

# S.I.G.S.M

## Programación Full Stack

Empre S.A.

| Rol         | Apellido | Nombre | C.I       | Email                       |
|-------------|----------|--------|-----------|-----------------------------|
| Coordinador | Cabrera  | Martin | 4919741-2 | empre.sa.utu.isbo@gmail.com |

**Docente: Acosta Alvez, Fabian**

**Fecha de  
culminación  
24/06/2026**

**PRIMERA ENTREGA**



|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>1. Introducción.....</b>                                                  | <b>8</b>  |
| <b>1.1 Presentación del proyecto S.I.G.S.M.....</b>                          | <b>8</b>  |
| 1.2 Descripción general de los módulos del sistema.....                      | 8         |
| 1.3 Objetivo del documento.....                                              | 10        |
| <b>2. Justificación tecnológica.....</b>                                     | <b>11</b> |
| 2.1 Criterios utilizados para la selección del stack tecnológico.....        | 11        |
| 2.2 PHP como lenguaje backend.....                                           | 12        |
| 2.3 MySQL / MariaDB como sistema de gestión de base de datos.....            | 13        |
| 2.4 React como tecnología frontend.....                                      | 14        |
| 2.5 Material UI como librería de componentes visuales.....                   | 15        |
| 2.6 Vite como herramienta de construcción del frontend.....                  | 16        |
| 2.7 Node.js como requisito para el entorno de desarrollo.....                | 17        |
| 2.8 Git y GitHub como herramientas de control de versiones.....              | 18        |
| 2.9 Visual Studio Code como entorno de desarrollo.....                       | 19        |
| 2.10 Relación entre tecnologías seleccionadas y necesidades del sistema..... | 20        |
| 2.10.1 Acceso desde dispositivos móviles mediante QR.....                    | 20        |
| 2.10.2 Gestión administrativa del hospital.....                              | 20        |
| 2.10.3 Persistencia de datos de documentos, encuestas y traslados.....       | 21        |
| 2.10.4 Trabajo colaborativo del equipo.....                                  | 21        |
| 2.11 Conclusión de la justificación tecnológica.....                         | 21        |
| <b>3. Prototipos de interfaz.....</b>                                        | <b>23</b> |
| 3.1 Objetivo del prototipo.....                                              | 23        |
| 3.2 Herramienta utilizada para el diseño del prototipo.....                  | 24        |
| 3.3 Criterios generales de diseño.....                                       | 24        |



|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 3.4 Prototipo del módulo de documentación.....                   | 25        |
| 3.4.1 Panel de administración de documentos.....                 | 26        |
| 3.4.2 Vista de documento accesible mediante QR.....              | 27        |
| 3.5 Prototipo del módulo de ambulancias.....                     | 29        |
| 3.5.1 Panel de gestión de traslados.....                         | 29        |
| 3.5.2 Formulario de alta de traslado.....                        | 30        |
| 3.5.3 Vista de seguimiento del estado del traslado.....          | 32        |
| 3.6 Enlace al prototipo.....                                     | 33        |
| <b>4. Implementación de diseño responsivo.....</b>               | <b>33</b> |
| 4.1 Importancia del diseño responsivo.....                       | 33        |
| 4.2 Diseño orientado a funcionarios administrativos.....         | 34        |
| 4.3 Diseño orientado a pacientes desde dispositivos móviles..... | 34        |
| 4.4 Uso de React y Material UI para interfaces responsivas.....  | 35        |
| 4.5 Uso de Flexbox, Grid y componentes adaptables.....           | 35        |
| 4.6 Responsividad del módulo de documentación.....               | 36        |
| 4.7 Responsividad del módulo de ambulancias.....                 | 37        |
| 4.8 Criterios de calidad visual.....                             | 37        |
| 4.9 Evidencias.....                                              | 37        |
| <b>5. Modelado de datos.....</b>                                 | <b>38</b> |
| 5.1 Introducción al modelado de datos del sistema.....           | 38        |
| 5.2 Modelo Entidad-Relación inicial.....                         | 39        |
| 5.2.1 Entidades principales del módulo de documentación.....     | 40        |
| 5.2.2 Entidades principales del módulo de ambulancias.....       | 45        |
| 5.2.3 Representación visual de las entidades y tablas.....       | 48        |



|                                                         |    |
|---------------------------------------------------------|----|
| 5.3 Restricciones no estructurales.....                 | 51 |
| 5.3.1 Restricciones del módulo de documentación.....    | 51 |
| 5.3.2 Restricciones del módulo de ambulancias.....      | 55 |
| 5.3.3 Forma de implementación de las restricciones..... | 57 |
| 5.4 Esquema relacional normalizado.....                 | 58 |
| 5.4.1 Pasaje del DER al modelo relacional.....          | 59 |
| 5.4.2 Tabla funcionario.....                            | 60 |
| 5.4.3 Tabla rol.....                                    | 60 |
| 5.4.4 Tabla rol_usuario.....                            | 61 |
| 5.4.5 Tabla categoria.....                              | 61 |
| 5.4.6 Tabla documento.....                              | 62 |
| 5.4.7 Tabla QR.....                                     | 62 |
| 5.4.8 Tabla encuesta.....                               | 63 |
| 5.4.9 Tabla pregunta.....                               | 64 |
| 5.4.10 Tabla opcion_respuesta.....                      | 65 |
| 5.4.11 Tabla respuesta_encuesta.....                    | 65 |
| 5.4.12 Tabla detalle_respuesta.....                     | 66 |
| 5.4.13 Tabla vehículo.....                              | 66 |
| 5.4.14 Tabla estado_traslado.....                       | 67 |
| 5.4.15 Tabla traslado.....                              | 67 |
| 5.4.16 Tabla historial_estado_traslado.....             | 68 |
| 5.4.17 Identificación de claves primarias.....          | 69 |
| 5.4.18 Identificación de claves foráneas.....           | 70 |
| 5.4.19 Normalización hasta tercera forma normal.....    | 71 |



|                                                                |           |
|----------------------------------------------------------------|-----------|
| 5.4.20 Modificaciones realizadas durante la normalización..... | 74        |
| 5.4.21 Justificación de decisiones de diseño.....              | 75        |
| <b>6. Configuración del entorno de desarrollo.....</b>         | <b>77</b> |
| 6.1 Objetivo del entorno de desarrollo.....                    | 77        |
| 6.2 Requisitos de software.....                                | 78        |
| 6.2.1 Sistema operativo.....                                   | 78        |
| 6.2.2 XAMPP 8.2.12 / Apache.....                               | 78        |
| 6.2.3 PHP.....                                                 | 79        |
| 6.2.4 MariaDB/MySQL.....                                       | 79        |
| 6.2.5 Node.js y npm.....                                       | 80        |
| 6.2.6 Git.....                                                 | 81        |
| 6.2.7 Visual Studio Code.....                                  | 82        |
| 6.2.8 Navegador web actualizado.....                           | 82        |
| 6.2.9 VirtualBox o VMware para servidor de pruebas.....        | 83        |
| 6.3 Creación de la máquina virtual.....                        | 84        |
| 6.4 Instalación de Fedora Server.....                          | 86        |
| 6.5 Configuración de red inicial.....                          | 90        |
| 6.6 Configuración básica de SSH.....                           | 91        |
| 6.7 Configuración básica del firewall.....                     | 92        |
| 6.8 Instalación de XAMPP.....                                  | 93        |
| 6.9 Instalación y configuración de Node.js.....                | 94        |
| 6.10 Configuración de Visual Studio Code.....                  | 95        |
| 6.10.1 PHP Intelephense.....                                   | 95        |
| 6.10.2 GitLens.....                                            | 96        |



|                                                  |            |
|--------------------------------------------------|------------|
| 6.10.3 Prettier.....                             | 96         |
| 6.10.4 Extensión MySQL o Database Client.....    | 96         |
| 6.10.5 Extensiones recomendadas para React.....  | 97         |
| 6.11 Verificación del entorno de desarrollo..... | 97         |
| 6.11.1 Verificación de Apache.....               | 97         |
| 6.11.2 Verificación de PHP.....                  | 98         |
| 6.11.3 Verificación de MariaDB.....              | 98         |
| 6.11.4 Verificación de Node.js.....              | 99         |
| 6.11.5 Verificación de Git.....                  | 100        |
| 6.12 Clonado del repositorio del proyecto.....   | 100        |
| 6.13 Pruebas y evidencias de la instalación..... | 101        |
| <b>7. Conclusión.....</b>                        | <b>106</b> |
| <b>8. Bibliografía.....</b>                      | <b>108</b> |



Empre S.A.

Montevideo, 02 de junio de 2026

Fabian Acosta Alvez  
Asignatura: Programación Full Stack  
Instituto Superior Brazo Oriental

Presente.

A continuación, el alumno de tercero 3MJ del turno vespertino del Instituto Superior Brazo Oriental se presenta ante usted, con el fin de informar la creación del grupo Empre S.A. El correspondiente integrante con su rol es el siguiente:

A continuación, se detalla dicha integración y roles del grupo:

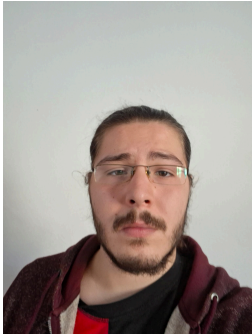
| ROL         | C.I       | APELLIDO | NOMBRE | E-MAIL               |
|-------------|-----------|----------|--------|----------------------|
| Coordinador | 4919741-2 | Cabrera  | Martin | mdcabrera@hotmail.es |

Por contacto al correo: [empre.sa.utu.isbo@gmail.com](mailto:empre.sa.utu.isbo@gmail.com)

Firma:

---

COORDINADOR



Cabrera, Martin

---

COORDINADOR





## **1. Introducción**

### **1.1 Presentación del proyecto S.I.G.S.M.**

El presente documento corresponde a la primera entrega de la asignatura Programación Full Stack, en el marco del Proyecto de Egreso ISBO 2026. El proyecto a desarrollar se denomina S.I.G.S.M., Sistema Informático de Gestión de Servicios Médicos, y surge a partir de la solicitud realizada por el Hospital de Clínicas.

La propuesta consiste en el desarrollo de una solución web orientada a mejorar determinados procesos administrativos y operativos dentro de la institución. El sistema estará compuesto por módulos independientes, pensados para integrarse a la infraestructura tecnológica existente del hospital y ser utilizados principalmente por funcionarios administrativos del área de la salud.

Desde el área de Programación Full Stack, el proyecto implica analizar, diseñar y preparar las bases técnicas necesarias para el posterior desarrollo de la aplicación. Para ello, se seleccionará un conjunto de tecnologías que permita construir una solución funcional, organizada, mantenible y compatible con los requerimientos planteados por el cliente.

### **1.2 Descripción general de los módulos del sistema**

El sistema S.I.G.S.M. estará compuesto por dos módulos principales: el módulo de gestión y acceso digital a documentación para pacientes, y el módulo de gestión y seguimiento de transporte de ambulancias.



El módulo de documentación permitirá que los funcionarios administrativos carguen, organicen y gestionen documentos informativos destinados a los pacientes del Hospital de Clínicas. Estos documentos estarán disponibles de forma digital y podrán ser consultados mediante el escaneo de un código QR desde dispositivos móviles. De esta manera, se busca facilitar el acceso a la información, reducir el uso de documentación impresa y mejorar la disponibilidad de los materiales informativos para los usuarios.

Dentro de este módulo también se contempla la incorporación de encuestas de satisfacción. Estas encuestas podrán ser completadas por los pacientes de forma digital y sus respuestas serán almacenadas para permitir posteriores análisis e indicadores relacionados con la calidad de los servicios brindados por la institución.

El módulo de ambulancias estará orientado a la gestión y trazabilidad de traslados realizados mediante ambulancias u otros vehículos del hospital. Permitirá registrar solicitudes de traslado, asociar conductores, enfermeros, pacientes o elementos a trasladar, definir origen y destino, controlar horarios y visualizar el estado del recorrido. Su objetivo principal es brindar al personal administrativo una herramienta que permita realizar un seguimiento ordenado de cada traslado, desde la salida del hospital hasta la llegada al destino y el posterior retorno.

Ambos módulos responden a necesidades distintas, pero comparten una misma finalidad: mejorar la gestión de servicios médicos mediante herramientas digitales que permitan organizar la información, facilitar el acceso a los datos y apoyar el trabajo administrativo dentro del Hospital de Clínicas.

### **1.3 Objetivo del documento**

El objetivo de este documento es presentar los elementos correspondientes a la primera entrega de Programación Full Stack, estableciendo las bases técnicas iniciales para el desarrollo del sistema S.I.G.S.M.

En primer lugar, se realizará una justificación tecnológica del stack seleccionado, explicando por qué se utilizarán determinadas herramientas para el backend, la base de datos, el frontend, el entorno de desarrollo y el control de versiones. Esta justificación no se limitará a describir cada tecnología, sino que buscará relacionarlas con necesidades concretas del sistema, como el acceso móvil mediante QR, la gestión administrativa y la persistencia de datos.

También se presentará el prototipo de interfaz de usuario, con el objetivo de representar visualmente las principales pantallas de ambos módulos antes de avanzar con su implementación completa. A su vez, se documentará la propuesta de diseño responsivo, considerando que el sistema deberá adaptarse tanto a equipos de escritorio utilizados por funcionarios como a dispositivos móviles utilizados por pacientes.

Otro objetivo del documento es incluir el modelado inicial de datos, mediante un Diagrama Entidad-Relación, la identificación de restricciones no estructurales y la transformación del modelo hacia un esquema relacional normalizado. Esto permitirá definir de forma ordenada cómo se almacenará la información relacionada con documentos, encuestas, usuarios, traslados, vehículos y estados.

Finalmente, este documento incluirá la configuración del entorno de desarrollo y la organización inicial del repositorio. De esta forma, se busca dejar establecido cómo



se preparará el ambiente de trabajo, qué herramientas serán necesarias, cómo se verificará su funcionamiento y qué criterios se utilizarán para mantener un control de versiones claro y organizado durante el desarrollo del proyecto.

## **2. Justificación tecnológica**

### **2.1 Criterios utilizados para la selección del stack tecnológico**

La selección de tecnologías para el desarrollo del proyecto S.I.G.S.M. se realiza teniendo en cuenta las necesidades reales del sistema, el entorno donde será implementado y los requerimientos planteados por el Hospital de Clínicas.

El sistema deberá funcionar como una solución web compuesta por dos módulos principales: un módulo de gestión y acceso digital a documentación para pacientes mediante códigos QR, y un módulo de gestión y seguimiento de transporte de ambulancias. Ambos módulos deberán integrarse a la infraestructura tecnológica del hospital, por lo que resulta necesario utilizar herramientas compatibles con entornos institucionales, servidores GNU/Linux, bases de datos relacionales y desarrollo web modular.

Para la selección del stack tecnológico se consideraron los siguientes criterios: compatibilidad con la infraestructura del hospital, facilidad de mantenimiento, organización del código, posibilidad de trabajo colaborativo, soporte para interfaces responsivas, persistencia segura de datos y separación clara entre frontend, backend y base de datos.

A partir de estos criterios, se propone el uso de PHP como lenguaje backend, MySQL/MariaDB como sistema de gestión de base de datos, React como tecnología frontend, Material UI como librería de componentes visuales, Vite como herramienta de construcción del frontend, Node.js como requisito del entorno de desarrollo, Git/GitHub para control de versiones y Visual Studio Code como entorno principal de desarrollo.

## **2.2 PHP como lenguaje backend**

Se selecciona PHP como lenguaje principal para el backend debido a su amplia compatibilidad con servidores GNU/Linux, su madurez como tecnología web y su uso frecuente en sistemas institucionales. (Pokoyny, n.d.) Esta elección resulta adecuada porque el proyecto deberá funcionar en los servidores del Departamento Técnico de Informática del Hospital de Clínicas, por lo que es importante utilizar una tecnología estable, conocida y compatible con entornos de producción tradicionales.

PHP permitirá desarrollar la lógica interna de ambos módulos. En el módulo de documentación será utilizado para gestionar la carga, edición, categorización y consulta de documentos informativos. También permitirá recuperar el contenido correspondiente cuando un paciente acceda a un documento mediante el escaneo de un código QR desde su dispositivo móvil.

En el módulo de ambulancias, PHP permitirá gestionar solicitudes de traslado, registrar conductores, enfermeros, vehículos, pacientes o elementos transportados, puntos de origen, destinos, horarios y estados del recorrido. Además, será el encargado de validar los datos ingresados antes de almacenarlos, evitando errores como solicitudes incompletas, datos inconsistentes u horarios incorrectos.

Otra ventaja importante de PHP es su integración directa con MySQL/MariaDB, lo que simplifica la comunicación entre la aplicación y la base de datos. (Pokoyny, n.d.) Esto resulta especialmente útil en un sistema que debe manejar información persistente y estructurada, como documentos, encuestas, usuarios, vehículos y trazabilidad de traslados.

Además, PHP cuenta con una amplia documentación, comunidad activa y gran cantidad de recursos de aprendizaje, lo que facilita la resolución de problemas durante el desarrollo. (Pokoyny, n.d.) Esto resulta beneficioso para el equipo, ya que permite consultar referencias confiables al momento de implementar funciones, conectar la base de datos, gestionar sesiones, validar formularios o resolver errores técnicos.

### **2.3 MySQL / MariaDB como sistema de gestión de base de datos**

Se propone el uso de MySQL o MariaDB como sistema de gestión de base de datos relacional. Esta elección se justifica porque el sistema necesita almacenar información de forma permanente, ordenada y consistente.

En el módulo de documentación, la base de datos permitirá almacenar documentos, categorías, usuarios administrativos, encuestas, preguntas y respuestas. En el módulo de ambulancias permitirá registrar solicitudes de traslado, conductores, enfermeros, vehículos, rutas, horarios, estados y datos asociados al seguimiento operativo de cada traslado.

El uso de una base de datos relacional es adecuado porque permite organizar la información en tablas relacionadas entre sí mediante claves primarias y claves foráneas. (MariaDB, 2026) Esto ayuda a evitar duplicación de datos, reducir inconsistencias y mantener la coherencia de la información. Por ejemplo, una

solicitud de traslado debe estar asociada a un vehículo válido, a un conductor, a un enfermero y a un estado concreto del recorrido. Del mismo modo, una encuesta debe conservar sus respuestas sin asociarlas a datos personales cuando se trate de una encuesta anónima.

Además, MySQL/MariaDB permite trabajar con un modelo de datos normalizado, lo cual favorece una estructura más limpia, mantenible y eficiente. La normalización ayuda a separar correctamente las entidades del sistema, evitar datos repetidos y facilitar futuras consultas, modificaciones o ampliaciones de la base de datos. (MariaDB, 2026)

Por estas razones, MySQL/MariaDB resulta adecuado para la persistencia de datos clínicos, administrativos y operativos relacionados con el funcionamiento del sistema.

## **2.4 React como tecnología frontend**

Se utilizará React como tecnología principal para el desarrollo de la interfaz de usuario. React es una librería de JavaScript orientada a la construcción de interfaces mediante componentes reutilizables, lo que resulta especialmente útil en un sistema dividido en módulos, pantallas administrativas, formularios, listados y vistas de seguimiento (React, n.d.).

Esta elección permite organizar el frontend de forma modular. Por ejemplo, se podrán crear componentes reutilizables para tarjetas informativas, tablas de documentos, formularios de traslado, formularios de encuesta, botones de acción, menús laterales y vistas de estado. Esto facilita el mantenimiento del sistema, ya que una modificación visual o funcional puede realizarse sobre un componente y reutilizarse en distintas pantallas.

Para las interfaces administrativas, React permitirá construir paneles dinámicos y organizados para los funcionarios del hospital. Esto será aplicado en el panel de documentos, la gestión de categorías, el módulo de encuestas, el panel de traslados y la vista de seguimiento de ambulancias.

Para las interfaces destinadas a pacientes, React permitirá desarrollar vistas simples y adaptadas a dispositivos móviles. Esto es fundamental para el acceso mediante códigos QR, ya que los pacientes accederán principalmente desde sus teléfonos. La vista de documentos y los formularios de encuesta deberán ser claros, accesibles y legibles en pantallas pequeñas.

Por lo tanto, React se selecciona porque permite desarrollar una interfaz moderna, modular y mantenible, adecuada para un sistema web institucional con distintos tipos de usuarios y necesidades de interacción.

## **2.5 Material UI como librería de componentes visuales**

Material UI será utilizado como librería de componentes visuales sobre React. Esta herramienta ofrece componentes previamente diseñados y consistentes, como botones, campos de texto, tablas, tarjetas, menús, alertas, diálogos y elementos de navegación. (Material UI, n.d.)

Su uso permitirá construir una interfaz profesional, clara y uniforme, manteniendo coherencia visual entre los distintos módulos del sistema. Esto es importante porque S.I.G.S.M. será utilizado por funcionarios administrativos del hospital y también por pacientes que accederán a determinadas vistas desde sus teléfonos.

En el módulo de documentación, Material UI facilitará la creación de pantallas para listar documentos, cargar nuevos archivos, organizar categorías, visualizar





información y completar encuestas. En el módulo de ambulancias, permitirá construir formularios ordenados, tablas de seguimiento, tarjetas de estado y paneles de control para los traslados.

Además, Material UI permite trabajar con diseño responsivo, adaptando los componentes a distintos tamaños de pantalla. Esto resulta fundamental para un sistema que deberá funcionar tanto en equipos de escritorio como en dispositivos móviles. (Material UI, n.d.)

La elección de Material UI también favorece la consistencia visual del proyecto, ya que permite mantener una misma línea de diseño en botones, formularios, tablas y elementos de navegación. Esto mejora la experiencia de usuario y facilita que las pantallas sean más comprensibles para quienes interactúan con el sistema.

## **2.6 Vite como herramienta de construcción del frontend**

El frontend del sistema se desarrollará utilizando Vite como herramienta de construcción para React. Vite permite crear proyectos frontend modernos de forma rápida, ordenada y eficiente, ofreciendo un entorno de desarrollo ágil para trabajar con componentes, estilos y recursos visuales. (Vite.dev, n.d.)

Una de las ventajas principales de Vite es que facilita la creación de una estructura inicial clara para el proyecto, permitiendo separar componentes, páginas, estilos y servicios de conexión con el backend. (Vite.dev, n.d.) Esto resulta útil para mantener el código organizado desde las primeras etapas del desarrollo.

Además, Vite permite separar la parte frontend de la aplicación del backend desarrollado en PHP. Esta separación resulta conveniente para el proyecto, ya que permite trabajar ambos lados del sistema de forma más ordenada: React se encarga



de la interfaz de usuario y PHP se encarga de la lógica interna, las validaciones y la comunicación con la base de datos. (Vite.dev, n.d.)

Durante el desarrollo, Vite permitirá ejecutar el frontend en un servidor local, visualizar los cambios rápidamente y trabajar de forma más cómoda sobre las distintas pantallas del sistema. Esto facilitará el prototipado y la implementación progresiva de las interfaces, tanto del módulo de documentación como del módulo de ambulancias.

Finalmente, Vite permite generar una versión final optimizada del frontend para su posterior despliegue. Esta versión podrá integrarse con el backend desarrollado en PHP, manteniendo separada la capa visual de la lógica del servidor. De esta forma, el sistema podrá contar con una arquitectura más ordenada, mantenible y preparada para futuras etapas de instalación o despliegue.

## **2.7 Node.js como requisito para el entorno de desarrollo**

Se utilizará Node.js como requisito del entorno de desarrollo frontend, ya que React, Material UI y Vite dependen del ecosistema de paquetes de JavaScript para funcionar correctamente durante la etapa de desarrollo.

Node.js no será utilizado como backend principal del sistema, ya que esa función será cumplida por PHP. Sin embargo, será necesario para instalar dependencias, ejecutar scripts del proyecto, levantar el servidor local de desarrollo de Vite y generar la versión final optimizada del frontend. (Node.js, n.d.)

Mediante Node.js y npm se podrán instalar las librerías necesarias para el desarrollo de la interfaz, como React, Material UI, Vite y otras dependencias complementarias.

También permitirá ejecutar comandos como la instalación de paquetes, el inicio del entorno de desarrollo y la generación del build final. (Node.js, n.d.)

Su incorporación resulta importante porque sin Node.js no sería posible levantar correctamente el frontend desarrollado con React durante la etapa de implementación. Por esta razón, Node.js será documentado como parte de los requisitos del entorno de desarrollo, aunque no forme parte del backend productivo del sistema.

Además, el uso de Node.js favorece la separación entre frontend y backend. Por un lado, PHP se encargará de la lógica del servidor y la conexión con MySQL/MariaDB; por otro lado, Node.js permitirá gestionar el entorno de desarrollo del frontend React. Esta separación facilita el trabajo del equipo, mejora la organización del proyecto y permite preparar una estructura compatible con futuras configuraciones mediante Docker.

## **2.8 Git y GitHub como herramientas de control de versiones**

Se utilizará Git como sistema de control de versiones y GitHub como plataforma para alojar el repositorio del proyecto. Esta elección es necesaria para mantener un historial claro de los cambios realizados por el equipo de trabajo.

Git permite registrar cada modificación del código, identificar quién realizó cada cambio y volver a versiones anteriores en caso de error. (Git, n.d.) Esto resulta imprescindible durante las distintas etapas del desarrollo, ya que el sistema irá incorporando correcciones, mejoras y nuevas funcionalidades.

Por otra parte, GitHub permitirá centralizar el código fuente, la documentación técnica, los prototipos y los scripts de base de datos. Además, facilita el trabajo



colaborativo mediante ramas, commits, historial de cambios y un archivo README con instrucciones de instalación y uso.

El uso de Git y GitHub también responde a buenas prácticas profesionales de desarrollo de software, ya que permite mantener un flujo de trabajo más ordenado, transparente y controlado. Para este proyecto, el repositorio será utilizado como espacio común de trabajo, documentación y seguimiento técnico del avance realizado.

## **2.9 Visual Studio Code como entorno de desarrollo**

Se recomienda Visual Studio Code como entorno principal de desarrollo debido a que es liviano, gratuito, flexible y compatible con las tecnologías seleccionadas. Además, permite trabajar con PHP, JavaScript, React, Material UI, bases de datos y Git desde un mismo espacio de trabajo.

Para este proyecto se considera conveniente utilizar extensiones como PHP Intelephense, para autocompletado y detección de errores en PHP; GitLens, para consultar el historial de commits; Prettier, para mantener un formato de código uniforme; y una extensión de MySQL o Database Client, para visualizar y consultar la base de datos desde el editor.

También se recomiendan extensiones orientadas a React y JavaScript, como “js jsx Snippets” o ESLint, ya que facilitarán el desarrollo de componentes, la detección de errores y el mantenimiento del frontend. Además, se podrá utilizar alguna extensión de conexión remota, como SSH FS u otra alternativa similar, en caso de necesitar trabajar sobre una máquina virtual o servidor de pruebas.

El uso de Visual Studio Code también facilita que todos los integrantes del equipo trabajen con una configuración similar, reduciendo errores por diferencias de entorno y permitiendo documentar de forma clara el proceso de instalación y configuración.

## **2.10 Relación entre tecnologías seleccionadas y necesidades del sistema**

### **2.10.1 Acceso desde dispositivos móviles mediante QR**

Para el acceso desde dispositivos móviles mediante QR se utilizará React junto con Material UI, permitiendo crear vistas responsivas, simples y legibles para los pacientes. Esto es fundamental porque los usuarios accederán principalmente desde sus teléfonos al escanear un código QR.

PHP será el encargado de recuperar dinámicamente el documento correspondiente desde la base de datos y servir la información necesaria para que el paciente pueda visualizarla. MySQL/MariaDB permitirá almacenar los documentos, categorías y datos asociados al contenido disponible.

### **2.10.2 Gestión administrativa del hospital**

Para la gestión administrativa del hospital, React y Material UI permitirán construir paneles de control, formularios, tablas y vistas de seguimiento claras para los funcionarios. Estas interfaces serán utilizadas para administrar documentos, cargar información, gestionar encuestas y realizar el seguimiento de traslados.

PHP se encargará de procesar las operaciones internas del sistema, validar los datos ingresados y comunicarse con la base de datos. De esta forma, el frontend y el backend trabajarán de manera separada pero coordinada.

### **2.10.3 Persistencia de datos de documentos, encuestas y traslados**

Para la persistencia de datos se utilizará MySQL/MariaDB, ya que permite almacenar la información de forma estructurada mediante tablas relacionadas. Esto será aplicado a documentos, categorías, encuestas, respuestas, usuarios, vehículos, conductores, enfermeros, solicitudes de traslado y estados del recorrido.

El uso de claves primarias, claves foráneas y relaciones entre tablas permitirá mantener la integridad y consistencia de la información, reduciendo duplicaciones y evitando inconsistencias entre los datos registrados.

### **2.10.4 Trabajo colaborativo del equipo**

Para el trabajo colaborativo del equipo se utilizarán Git y GitHub. Estas herramientas permitirán organizar el avance del proyecto, registrar cambios, trabajar mediante ramas, documentar decisiones y mantener un repositorio estructurado.

Además, Visual Studio Code facilitará que se trabaje con un entorno común y replicable, mientras que el README del repositorio permitirá documentar instrucciones de instalación, ejecución y uso del sistema.

## **2.11 Conclusión de la justificación tecnológica**

La elección de las tecnologías mencionadas resulta adecuada para el desarrollo del proyecto S.I.G.S.M., ya que responde a los requerimientos técnicos y a las necesidades planteadas por el Hospital de Clínicas. El stack seleccionado permite

construir una solución web institucional, organizada, mantenible y compatible con un entorno basado en servidores GNU/Linux.

PHP aporta una base sólida para desarrollar la lógica del sistema, gestionar formularios, validar información, controlar operaciones internas y conectar la aplicación con la base de datos. MySQL/MariaDB permite almacenar de forma estructurada y persistente los datos relacionados con documentos, encuestas, usuarios, traslados, vehículos y estados de seguimiento, manteniendo la integridad y consistencia de la información.

Por otra parte, React y Material UI permiten desarrollar una interfaz moderna, modular y reutilizable. React facilita la construcción de componentes independientes para formularios, tablas, tarjetas, paneles administrativos y vistas móviles, mientras que Material UI aporta componentes visuales consistentes y profesionales. Esta combinación resulta adecuada para un sistema que será utilizado tanto por funcionarios administrativos desde equipos de escritorio como por pacientes desde dispositivos móviles mediante códigos QR.

Vite y Node.js cumplen un rol importante durante el desarrollo del frontend. Vite permite trabajar con React de forma rápida y ordenada, mientras que Node.js es necesario para instalar dependencias, ejecutar scripts y levantar el entorno local de desarrollo. De esta manera, el equipo podrá probar las interfaces, realizar ajustes visuales y generar una versión final optimizada del frontend para su posterior integración con el backend en PHP.

Git y GitHub permiten organizar el trabajo colaborativo, registrar los avances del proyecto y mantener un control ordenado de las modificaciones realizadas. A su vez, Visual Studio Code proporciona un entorno de desarrollo práctico y flexible,



compatible con las herramientas seleccionadas y útil para mantener una estructura de trabajo común.

En conjunto, el stack tecnológico seleccionado permite desarrollar una aplicación web capaz de gestionar documentación digital para pacientes, encuestas de satisfacción y trazabilidad de traslados en ambulancia. Además, favorece la separación entre frontend, backend y base de datos, lo que facilita la documentación, el mantenimiento, el trabajo en equipo y la futura instalación del sistema dentro de la infraestructura del hospital.

### **3. Prototipos de interfaz**

#### **3.1 Objetivo del prototipo**

El objetivo del prototipo de interfaz de usuario es representar visualmente las principales pantallas del sistema antes de comenzar con su implementación real. Esta etapa permite definir la estructura general de las vistas, la organización de los elementos, la navegación entre pantallas y la experiencia de usuario esperada para cada módulo.

El prototipo no incluye lógica de programación ni conexión real con la base de datos, ya que su finalidad principal es mostrar cómo se visualizará el sistema y cómo interactuarán los usuarios con sus distintas secciones. De esta manera se facilita la validación temprana del diseño, permitiendo detectar posibles mejoras antes de avanzar hacia el desarrollo funcional.

En el caso del proyecto, el prototipo cumple un rol importante debido a que el sistema será utilizado por distintos tipos de usuarios. Por un lado, los funcionarios administrativos del hospital utilizarán paneles de gestión, formularios y vistas de



seguimiento. Por otro lado, los pacientes accederán principalmente desde dispositivos móviles mediante códigos QR, por lo que las pantallas destinadas a ellos deben ser simples, claras y legibles.

### **3.2 Herramienta utilizada para el diseño del prototipo**

Para la elaboración del prototipo se utilizó Figma, una herramienta de diseño colaborativo orientada a la creación de interfaces de usuario, prototipos visuales y flujos de navegación.

La elección de Figma resulta adecuada para esta etapa del proyecto, ya que permite diseñar pantallas sin necesidad de programar, organizar componentes visuales, definir estilos reutilizables y representar de forma clara la estructura de la aplicación. Además, facilita el trabajo colaborativo entre los integrantes del equipo, permitiendo que todos puedan visualizar, comentar y modificar las pantallas diseñadas.

Mediante Figma se realizaron las vistas principales de los módulos del sistema, considerando la distribución de elementos, la jerarquía visual, la paleta de colores, la tipografía y la coherencia general entre pantallas. Esto permite contar con una referencia visual clara para la etapa posterior de desarrollo en React y Material UI.

El prototipo también funciona como una guía para la implementación, ya que permite trasladar las ideas visuales a componentes reutilizables del frontend, como tablas, formularios, tarjetas, botones, menús y vistas de detalle.

### **3.3 Criterios generales de diseño**

Para el diseño del prototipo se consideraron criterios orientados a la claridad, la simplicidad, la coherencia visual y la facilidad de uso. Al tratarse de una aplicación



vinculada al ámbito hospitalario, se buscó una interfaz limpia, ordenada y profesional, evitando una sobrecarga innecesaria de elementos visuales.

Uno de los criterios principales fue diferenciar correctamente las pantallas administrativas de las pantallas destinadas a pacientes. Las vistas administrativas priorizan la organización de datos, el acceso rápido a acciones y la visualización de listados, mientras que las vistas para pacientes priorizan la lectura simple, el uso desde teléfono y la accesibilidad de la información.

También se tuvo en cuenta la consistencia visual entre ambos módulos. Aunque cada módulo cumple una función distinta, el sistema debe mantener una identidad común en cuanto a colores, tipografías, botones, tarjetas, formularios y estructura general. Esto permite que el usuario perciba la aplicación como un sistema integrado y no como pantallas aisladas.

La paleta de colores utilizada se orienta a transmitir confianza, claridad y relación con el ámbito médico. Se prioriza el uso de tonos azules y colores neutros, ya que estos se asocian frecuentemente con instituciones de salud, profesionalismo y seguridad. A su vez, los colores secundarios se utilizan para diferenciar estados, acciones y secciones específicas del sistema.

La tipografía seleccionada busca favorecer la legibilidad en distintos dispositivos. Por este motivo, se utilizan fuentes simples, modernas y claras, adecuadas tanto para pantallas de escritorio como para dispositivos móviles.

### **3.4 Prototipo del módulo de documentación**

El módulo de documentación está orientado a la carga, organización y consulta digital de documentos informativos destinados a pacientes del Hospital de Clínicas.



Este módulo contempla una parte administrativa, utilizada por funcionarios, y una parte pública o de consulta, utilizada por pacientes al escanear un código QR.

El diseño del prototipo busca que los funcionarios puedan gestionar la documentación de forma ordenada, mientras que los pacientes puedan acceder a la información de manera sencilla desde sus dispositivos móviles.

### **3.4.1 Panel de administración de documentos**

El panel de administración de documentos está diseñado para que el funcionario pueda visualizar, cargar, eliminar y organizar documentos informativos dentro del sistema.

La pantalla principal presenta un listado de documentos con información relevante, como nombre del documento, categoría, estado, fecha de carga y acciones disponibles. También se incluye una opción para agregar un nuevo documento, permitiendo que el usuario administrativo pueda iniciar el proceso de carga desde la misma vista.

El diseño del panel busca facilitar la búsqueda y gestión de documentos. Para ello, se contemplan elementos como campos de búsqueda, filtros por categoría, botones de acción y una estructura visual basada en tablas o tarjetas. Esta organización permite que el funcionario encuentre rápidamente la información necesaria y realice acciones sin recorrer múltiples pantallas innecesarias.

Además, el panel contempla la posibilidad de asociar cada documento a un código QR, permitiendo que posteriormente los pacientes puedan acceder al contenido desde sus teléfonos móviles.

## Panel de administración

Gestión de documentación clínica

Módulo de documentación

- Inicio
- Categorías
- Documentos**
- Encuestas

Módulo de ambulancias

- Inicio
- Traslados

Administrador  
admin@hc.com

Subir documento

Todas las categorías

Todos los estados

| Documento                                   | Categoría      | Fecha de subida | Estado | Acciones |
|---------------------------------------------|----------------|-----------------|--------|----------|
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |
| Protocolo terapéutico de uso de vasopresina | Anestesiología | 15/05/2026      | Activo |          |

<

1

2

3

4

>

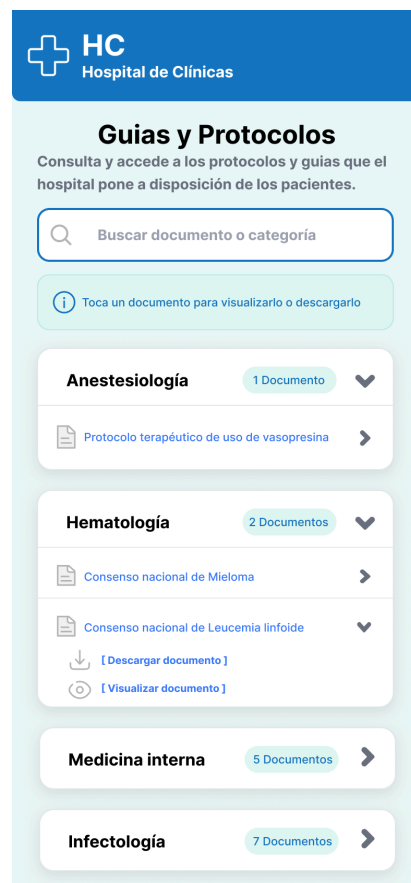
### 3.4.2 Vista de documento accesible mediante QR

La vista de documento accesible mediante QR está pensada principalmente para pacientes. Por este motivo, se diseñó con un enfoque mobile-first, priorizando la visualización desde dispositivos móviles.

Cuando el paciente escanea el código QR, accede a una pantalla simple donde puede buscar, descargar o visualizar el documento o la información correspondiente. Esta vista debe presentar el contenido de forma clara, con buena legibilidad, márgenes adecuados y una estructura que no requiera conocimientos técnicos por parte del usuario.

El diseño evita elementos administrativos o acciones innecesarias, ya que el objetivo principal es que el paciente pueda consultar la información de manera rápida y directa. En caso de que el documento pertenezca a una categoría específica, la pantalla podrá mostrar el nombre de la categoría, el título del documento y el contenido asociado.

Esta vista resulta fundamental dentro del sistema, ya que representa el punto de contacto directo entre el paciente y la documentación digital proporcionada por el Hospital de Clínicas.



### **3.5 Prototipo del módulo de ambulancias**

El módulo de ambulancias está orientado a la gestión y seguimiento de traslados realizados mediante ambulancias u otros vehículos del Hospital de Clínicas. Su diseño está pensado principalmente para funcionarios administrativos, quienes serán responsables de registrar solicitudes, consultar datos y actualizar estados de traslado.

El prototipo de este módulo busca representar una herramienta interna, clara y funcional, que permita visualizar rápidamente la situación de cada traslado y acceder a las acciones necesarias para su gestión.

#### **3.5.1 Panel de gestión de traslados**

El panel de gestión de traslados funciona como la vista principal del módulo de ambulancias. En esta pantalla el funcionario puede consultar el listado de traslados registrados, visualizar su estado actual y acceder a las acciones disponibles.

La pantalla presenta información relevante como identificador del traslado, conductor asignado, enfermero acompañante, paciente o elemento trasladado, origen, destino, horario de salida, horario estimado o efectivo de llegada y estado del recorrido.

El diseño del panel busca que el funcionario pueda identificar rápidamente qué traslados están pendientes, en curso, finalizados o en retorno. Para ello, se pueden utilizar etiquetas de estado, colores diferenciados y botones de acción como “ver detalle”, “actualizar estado” o “registrar nuevo traslado”.

Esta vista es importante porque permite centralizar el seguimiento operativo de las ambulancias y otros vehículos, facilitando el control administrativo de los traslados realizados por el hospital.

Módulo de documentación

- Inicio
- Categorías
- Documentos
- Encuestas

Módulo de ambulancias

- Inicio
- Traslados

Administrador  
admin@hrc.com

**Panel de administración**

**Traslados en curso**

Q Buscar traslado...

| Código       | Paciente / elemento                    | Origen               | Destino             | Hora de salida | Estado           | Prioridad | Acciones |
|--------------|----------------------------------------|----------------------|---------------------|----------------|------------------|-----------|----------|
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | En camino        | Urgente   |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Completado       | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Retornando       | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |
| TR-2026-0142 | Maria Gomez<br><small>Paciente</small> | Hospital de Clínicas | Islas Canarias 1234 | 10:30          | Llegó al destino | Normal    |          |

< 1 2 3 4 >

**Detalle del traslado**

- Paciente: **Maria Gomez**
- Conductor: **Laura Martinez**
- Ambulancia: **Ambulancia A-01**
- Origen: **Hospital de Clínicas**
- Destino: **Islas Canarias 123**
- Hora salida: **16/06/2026 - 10:30**
- Hora estimada de llegada: **11:45**

**Estado actual**

Traslado registrado en el sistema por Juan Perez

- Registrado**  
16/06/2026 - 10:25
- En camino**  
16/06/2026 - 10:30
- Llegó al destino**  
Pendiente
- Retornando**  
Pendiente
- Completado**  
Pendiente

### 3.5.2 Formulario de alta de traslado

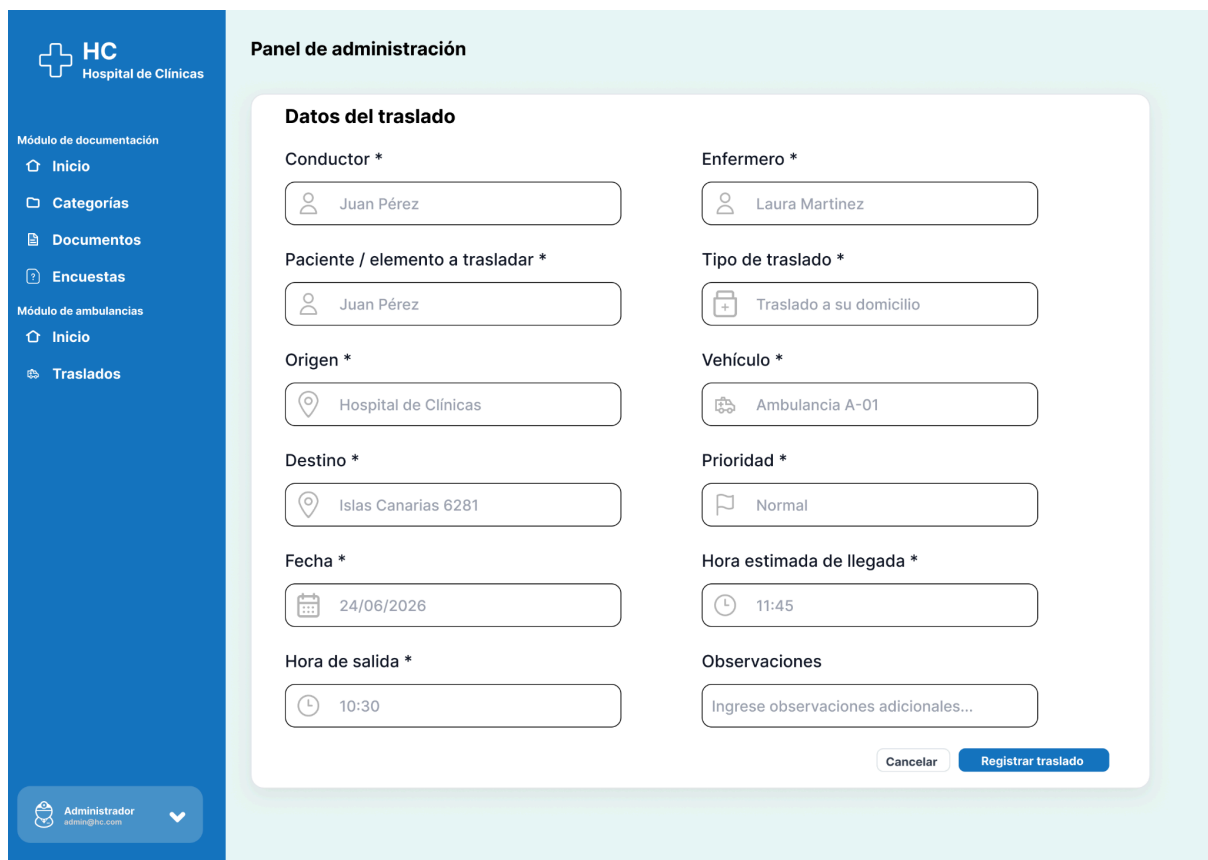
El formulario de alta de traslado está diseñado para registrar una nueva solicitud dentro del sistema. Esta pantalla permite ingresar los datos necesarios para iniciar el seguimiento de una operación de traslado.

Entre los campos contemplados se encuentran el conductor responsable, el enfermero acompañante, el paciente o elemento a trasladar, el tipo de traslado, el

vehículo asignado, el punto de origen, el destino, la hora de salida y la hora estimada de llegada.

El diseño del formulario prioriza el orden y la claridad, agrupando los campos de forma lógica para reducir errores durante la carga de datos. También se busca que los campos obligatorios sean fácilmente identificables y que la interfaz ayude al usuario a completar la información de manera correcta.

Este formulario representa una parte esencial del módulo, ya que a partir de los datos ingresados se podrá realizar el seguimiento posterior del traslado y consultar su estado durante todo el recorrido.



**HC**  
Hospital de Clínicas

Módulo de documentación

- Inicio
- Categorías
- Documentos
- Encuestas

Módulo de ambulancias

- Inicio
- Traslados

**Panel de administración**

**Datos del traslado**

|                                                   |                                                                   |
|---------------------------------------------------|-------------------------------------------------------------------|
| Conductor *                                       | Enfermero *                                                       |
| <input type="text" value="Juan Pérez"/>           | <input type="text" value="Laura Martinez"/>                       |
| Paciente / elemento a trasladar *                 | Tipo de traslado *                                                |
| <input type="text" value="Juan Pérez"/>           | <input type="text" value="Traslado a su domicilio"/>              |
| Origen *                                          | Vehículo *                                                        |
| <input type="text" value="Hospital de Clínicas"/> | <input type="text" value="Ambulancia A-01"/>                      |
| Destino *                                         | Prioridad *                                                       |
| <input type="text" value="Islas Canarias 6281"/>  | <input type="text" value="Normal"/>                               |
| Fecha *                                           | Hora estimada de llegada *                                        |
| <input type="text" value="24/06/2026"/>           | <input type="text" value="11:45"/>                                |
| Hora de salida *                                  | Observaciones                                                     |
| <input type="text" value="10:30"/>                | <input type="text" value="Ingrese observaciones adicionales..."/> |

Administrador  
admin@hc.com



### 3.5.3 Vista de seguimiento del estado del traslado

La vista de seguimiento del estado del traslado permite consultar el detalle de una solicitud específica y visualizar su evolución durante el recorrido.

Esta pantalla muestra la información principal del traslado, incluyendo conductor, enfermero, paciente o elemento trasladado, vehículo, origen, destino, horarios y estado actual. Además, puede incorporar una línea de tiempo o sección de estados donde se indiquen las distintas etapas del traslado, como pendiente, en camino, llegó al destino, retornando y completado.

El objetivo de esta vista es que el funcionario administrativo pueda conocer rápidamente en qué situación se encuentra cada traslado y actualizar su estado cuando corresponda. Esto permite mantener un registro ordenado del proceso desde la salida del hospital hasta el retorno del vehículo.

El diseño de la pantalla busca ser claro, directo y fácil de interpretar, ya que será utilizado en un contexto operativo donde la información debe consultarse de forma rápida.

| Detalle del traslado                                                                                         |                             | Estado actual                                                                                               |                                              |
|--------------------------------------------------------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------|----------------------------------------------|
|  Paciente                 | <b>Maria Gomez</b>          |  <b>Registrado</b>       | <b>Traslado registrado en el sistema por</b> |
|  Conductor                | <b>Laura Martínez</b>       |                                                                                                             | <b>Juan Pérez</b>                            |
|  Ambulancia               | <b>Ambulancia A-01</b>      |  <b>En camino</b>        |                                              |
|  Origen                   | <b>Hospital de Clínicas</b> |                                                                                                             |                                              |
|  Destino                  | <b>Islas Canarias 123</b>   |  <b>Llegó al destino</b> |                                              |
|  Hora salida              | <b>16/06/2026 - 10:30</b>   |                                                                                                             | <b>Pendiente</b>                             |
|  Hora estimada de llegada | <b>11:45</b>                |  <b>Retornando</b>       |                                              |
|                                                                                                              |                             |                                                                                                             | <b>Pendiente</b>                             |
|                                                                                                              |                             |  <b>Completado</b>       |                                              |
|                                                                                                              |                             |                                                                                                             | <b>Pendiente</b>                             |

### **3.6 Enlace al prototipo**

El prototipo interactivo del sistema fue desarrollado en Figma y permite visualizar la estructura general de las pantallas diseñadas para ambos módulos. A través de [este enlace](#) se podrá acceder al diseño completo, revisar la organización visual de la interfaz y observar la propuesta inicial de navegación entre pantallas.

## **4. Implementación de diseño responsivo**

### **4.1 Importancia del diseño responsivo**

El diseño responsivo es un aspecto fundamental dentro del sistema S.I.G.S.M., ya que la aplicación será utilizada desde distintos tipos de dispositivos. Por un lado, los funcionarios administrativos del Hospital de Clínicas accederán principalmente desde computadoras de escritorio o notebooks para gestionar documentos, encuestas y traslados. Por otro lado, los pacientes accederán desde dispositivos móviles al escanear códigos QR para consultar documentación o completar formularios de satisfacción.

Por este motivo, las interfaces del sistema deben adaptarse correctamente a diferentes tamaños de pantalla, evitando desbordamientos, contenido cortado o dificultades de lectura. La implementación responsiva busca que el usuario pueda interactuar con el sistema de forma clara y ordenada, independientemente del dispositivo utilizado.

## **4.2 Diseño orientado a funcionarios administrativos**

Las vistas destinadas a funcionarios administrativos fueron pensadas principalmente para pantallas de escritorio, ya que estos usuarios deberán trabajar con listados, formularios, tablas y paneles de control.

En estas pantallas se prioriza la organización de la información, el acceso rápido a acciones y la visualización clara de los datos. Por ejemplo, en el módulo de documentación se contempla un panel para gestionar documentos, categorías y encuestas. En el módulo de ambulancias se contempla un panel para visualizar traslados, consultar estados y registrar nuevas solicitudes.

Aunque estas vistas están orientadas principalmente a escritorio, también se busca que puedan adaptarse correctamente a tablets o pantallas más reducidas, reorganizando los elementos cuando sea necesario.

## **4.3 Diseño orientado a pacientes desde dispositivos móviles**

Las vistas destinadas a pacientes se diseñan con un enfoque mobile-first, ya que el acceso principal se realizará mediante el escaneo de códigos QR desde teléfonos celulares.

En este caso, se prioriza una interfaz simple, clara y legible. El paciente no necesita acceder a funciones administrativas, sino consultar documentación o completar una encuesta de satisfacción. Por este motivo, las pantallas móviles deben reducir la cantidad de elementos visibles, utilizar textos fáciles de leer y presentar botones o campos de formulario de forma cómoda para el uso táctil.



Este criterio se aplica especialmente en la vista de documento por QR y en el formulario de encuesta de satisfacción.

#### **4.4 Uso de React y Material UI para interfaces responsivas**

Para pasar los prototipos visuales a un escenario más cercano a la aplicación real, se comenzó a implementar la interfaz utilizando React junto con Vite y Material UI.

React permite dividir la interfaz en componentes reutilizables, como tarjetas, formularios, tablas, botones, menús y vistas de detalle. (React, n.d.) Esto facilita mantener una estructura ordenada y aplicar el mismo criterio visual en diferentes pantallas del sistema.

Material UI aporta componentes ya preparados para trabajar con interfaces responsivas, permitiendo adaptar la distribución de los elementos según el tamaño de pantalla. (Material UI, n.d.) Además, facilita mantener una estética uniforme entre los distintos módulos, lo cual es importante para que el sistema se perciba como una única aplicación integrada.

Vite se utiliza como herramienta de desarrollo para levantar el frontend en un entorno local, probar los cambios rápidamente y generar una versión optimizada de la interfaz. Esto permite transformar los prototipos realizados previamente en pantallas funcionales, más cercanas al resultado final del sistema.

#### **4.5 Uso de Flexbox, Grid y componentes adaptables**

Para organizar las pantallas se utilizarán técnicas de maquetado responsivo como Flexbox, Grid y componentes adaptables de Material UI. Estas herramientas



permiten distribuir los elementos de forma flexible, haciendo que las tarjetas, formularios, tablas y secciones cambien su disposición según el ancho disponible.

En pantallas grandes, los elementos pueden organizarse en varias columnas para aprovechar mejor el espacio. En pantallas pequeñas, los mismos elementos pueden apilarse verticalmente, facilitando la lectura y el uso desde dispositivos móviles.

Este enfoque permite que las vistas administrativas y las vistas para pacientes mantengan una estructura clara sin importar el dispositivo desde el cual se acceda.

#### **4.6 Responsividad del módulo de documentación**

En el módulo de documentación se busca que el panel administrativo funcione correctamente en escritorio, permitiendo visualizar documentos, categorías, estados y acciones disponibles. Las tablas o listados deberán adaptarse para evitar desbordamientos en pantallas más pequeñas.

La vista de documento por QR se implementará con un enfoque mobile-first, ya que está pensada para pacientes que acceden desde el celular. El contenido deberá mostrarse de forma simple, con textos legibles, márgenes adecuados y una navegación mínima.

El formulario de encuesta de satisfacción también se diseñará para uso móvil, priorizando campos claros, botones accesibles y una estructura sencilla para que el paciente pueda completar la encuesta sin dificultad.

## **4.7 Responsividad del módulo de ambulancias**

En el módulo de ambulancias, el panel de gestión de traslados estará orientado principalmente a funcionarios administrativos. Esta vista deberá permitir consultar información como conductor, enfermero, vehículo, origen, destino, horarios y estado del traslado.

El formulario de alta de traslado deberá adaptarse a escritorio y tablet, organizando los campos de forma clara para reducir errores durante la carga de datos. En pantallas pequeñas, los campos podrán reorganizarse en una sola columna.

La vista de seguimiento de estado deberá ser legible tanto en escritorio como en dispositivos móviles, permitiendo consultar rápidamente la situación de un traslado y sus principales datos operativos.

## **4.8 Criterios de calidad visual**

Para evaluar la calidad del diseño responsivo se tendrán en cuenta los siguientes criterios: que no existan desbordamientos de contenido, que los textos sean legibles, que los botones sean accesibles, que los formularios puedan completarse de forma cómoda y que la navegación sea clara.

También se buscará mantener una coherencia visual entre los módulos, utilizando una misma paleta de colores, tipografía, estilo de botones, tarjetas y formularios. Esto permitirá que el sistema mantenga una identidad visual uniforme.

## **4.9 Evidencias**

Como evidencia de la implementación responsiva se incluirá el enlace al repositorio de GitHub, donde se podrá consultar el código fuente del frontend desarrollado con

React, Vite y Material UI. Además se incluirá el enlace a GitHub Pages con la versión desplegada del código para facilitar la demostración de la implementación realizada.

Repositorio de GitHub: [MrGalletah/S.I.G.S.M---Proyecto-UTU: S.I.G.S.M. - Módulos para el Hospital de Clínicas](https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU: S.I.G.S.M. - Módulos para el Hospital de Clínicas).

GitHub Pages: <https://mrgalletah.github.io/prototipos-primera-entrega-pages/>

Para acceder correctamente al código alojado en GitHub, los docentes deberán haber aceptado previamente la invitación al repositorio privado. En caso contrario, no podrán visualizar el código ni las vistas Kanban y Roadmap asociadas al proyecto.

## **5. Modelado de datos**

### **5.1 Introducción al modelado de datos del sistema**

El modelado de datos del sistema S.I.G.S.M. tiene como objetivo representar de forma ordenada la información que será gestionada por la aplicación. Esta etapa permite identificar las entidades principales del sistema, sus atributos, las relaciones existentes entre ellas y las restricciones que deberán cumplirse para mantener la coherencia de la información.

El sistema se compone de dos módulos principales: el módulo de gestión y acceso digital a documentación para pacientes, y el módulo de gestión y seguimiento de transporte de ambulancias. Dentro del módulo de documentación también se contempla la gestión de encuestas, aunque en esta propuesta las encuestas no se

consideran un módulo totalmente independiente, sino una sección asociada a las categorías y a la organización general de la documentación.

El modelado de datos resulta fundamental porque permite definir una estructura clara antes de implementar la base de datos. A partir del análisis de los requerimientos se identifican los objetos relevantes del sistema, se establecen sus relaciones y se transforman posteriormente en tablas relacionales. Esto facilita la organización de los datos, evita duplicaciones innecesarias y permite mantener la integridad de la información mediante claves primarias, claves foráneas y restricciones.

En esta primera entrega se presenta un modelo inicial, sujeto a posibles ajustes en próximas etapas del proyecto. El objetivo no es cerrar definitivamente la base de datos, sino contar con una primera estructura coherente que represente las necesidades principales del sistema y sirva como base para el desarrollo posterior.

## **5.2 Modelo Entidad-Relación inicial**

El Modelo Entidad-Relación inicial representa las principales entidades necesarias para el funcionamiento del sistema S.I.G.S.M. y la forma en que estas se vinculan entre sí.

Para su elaboración se tuvieron en cuenta los requerimientos de los dos módulos solicitados por el Hospital de Clínicas. En el módulo de documentación se consideran especialmente las entidades Funcionario, Rol, Rol\_Usuario, Categoría, Documento y QR, ya que son las que estructuran la gestión de documentos y el acceso digital por parte de los pacientes. A su vez, la sección de encuestas se plantea como una extensión de este módulo, integrada a la lógica de categorías.





En el módulo de ambulancias se consideran entidades vinculadas a la gestión de traslados, vehículos, estados y ubicaciones. En este caso, la entidad Funcionario y la estructura de roles también se reutilizan, evitando duplicar el manejo de usuarios en distintas partes del sistema.

### 5.2.1 Entidades principales del módulo de documentación

Las entidades principales de este módulo se pueden consultar en el siguiente [enlace](#) y son:

#### **Funcionario**

Representa a los funcionarios que acceden al sistema para administrar documentación, categorías y otras secciones del sistema.

#### Atributos principales

- id\_func
- nombre
- correo
- pwd\_hash
- activo

#### **Rol**

Representa los tipos de rol que pueden asignarse a los funcionarios dentro del sistema.

#### Atributos principales

- id\_rol
- nombre



- descripcion

### **Rol\_Usuario**

Representa la asignación de roles a cada usuario. Se utiliza como entidad intermedia para resolver la relación entre funcionario y rol, permitiendo que un funcionario pueda tener uno o varios roles.

#### Atributos principales

- id\_rol
- id\_func
- fecha\_asignacion

### **Categoría**

Representa la clasificación general del contenido del módulo de documentación. Permite organizar tanto documentos como secciones relacionadas como las encuestas.

#### Atributos principales

- id\_cat
- nombre
- descripcion

### **Documento**

Representa cada documento informativo cargado en el sistema, los cuales podrán ser consultados digitalmente por pacientes

#### Atributos principales

- id\_doc



- id\_cat
- id\_func
- ruta
- fecha\_subida
- título
- estado
- descripcion

## **QR**

Representa el acceso mediante código QR al módulo de documentación. Se plantea como un acceso general a la aplicación.

### Atributos principales

- id\_qr
- url
- token
- estado

## **Encuesta**

Representa una encuesta de satisfacción o relevamiento asociada a una categoría del módulo de documentación. Se plantea como una sección integrada dentro de las categorías, permitiendo que una categoría pueda contener documentos informativos y también encuestas relacionadas con un servicio, área o actividad del hospital.

### Atributos principales



- id\_encuesta
- id\_cat
- id\_func
- título
- descripcion
- fecha\_creacion
- anónima
- estado

### **Pregunta**

Representa cada una de las preguntas que forman parte de una encuesta. Permite estructurar el contenido del formulario y definir qué información se desea obtener por parte del paciente o usuario.

#### Atributos principales

- id\_pregunta
- id\_encuesta
- texto
- tipo
- obligatoria
- orden

### **Opcion\_respuesta**



Representa las opciones disponibles para aquellas preguntas que sean de tipo cerrado o de selección. Esta entidad permite que una pregunta tenga varias respuestas posibles predefinidas.

#### Atributos principales

- id\_opcion
- id\_pregunta
- texto
- valor

#### **Respuesta\_Encuesta**

Representa una respuesta enviada por un usuario a una encuesta determinada. En caso de que la encuesta sea anónima, esta entidad no deberá almacenar datos personales identificatorios.

#### Atributos principales

- id\_respuesta\_encuesta
- id\_encuesta
- cedula
- fecha\_respuesta

#### **Detalle\_Respuesta**

Representa la respuesta concreta dada a cada pregunta dentro de una encuesta enviada.

#### Atributos principales



- id\_detalle\_respuesta
- id\_pregunta
- id\_respuesta\_encuesta
- id\_opcion
- respuesta

### 5.2.2 Entidades principales del módulo de ambulancias

Las entidades principales de este módulo se pueden consultar en el siguiente [enlace](#) y son:

#### **Funcionario**

Representa a los funcionarios que acceden al sistema para administrar traslados.

##### Atributos principales

- id\_func
- nombre
- correo
- pwd\_hash
- activo

#### **Rol**

Representa los tipos de rol que pueden asignarse a los funcionarios dentro del sistema.

##### Atributos principales

- id\_rol
- nombre



- descripcion

### **Rol\_Usuario**

Representa la asignación de roles a cada usuario. Se utiliza como entidad intermedia para resolver la relación entre funcionario y rol, permitiendo que un funcionario pueda tener uno o varios roles.

#### Atributos principales

- id\_rol
- id\_func
- fecha\_asignacion

### **Traslado**

Representa una solicitud de traslado.

#### Atributos principales

- id\_traslado
- fecha\_solicitud
- hora\_salida
- hora\_llegada
- hora\_llegada\_estimada
- elemento
- origen
- destino
- id\_conductor
- id\_enfermero
- id\_estado
- id\_func\_solicitante



- id\_vehiculo

### **Vehículo**

Representa los vehículos disponibles para realizar traslados.

Atributos principales

- id\_vehículo
- modelo
- tipo
- estado

### **Estado\_Traslado**

Representa los estados posibles de un traslado

Atributos principales

- id\_estado
- nombre
- orden
- descripcion

### **Historial\_estado\_traslado**

Representa los cambios de estado durante el ciclo de vida del traslado.

Atributos principales

- id\_historial
- id\_traslado
- fecha\_hora



- id\_estado
- observación
- id\_func

### 5.2.3 Representación visual de las entidades y tablas

Para complementar la descripción textual del modelo de datos, se realizaron representaciones visuales de las entidades y tablas principales del sistema utilizando la herramienta dbdiagram.io. Esta herramienta permite construir diagramas a partir de una estructura definida en código DBML, facilitando la visualización de tablas, atributos, claves primarias, claves foráneas y relaciones entre entidades.

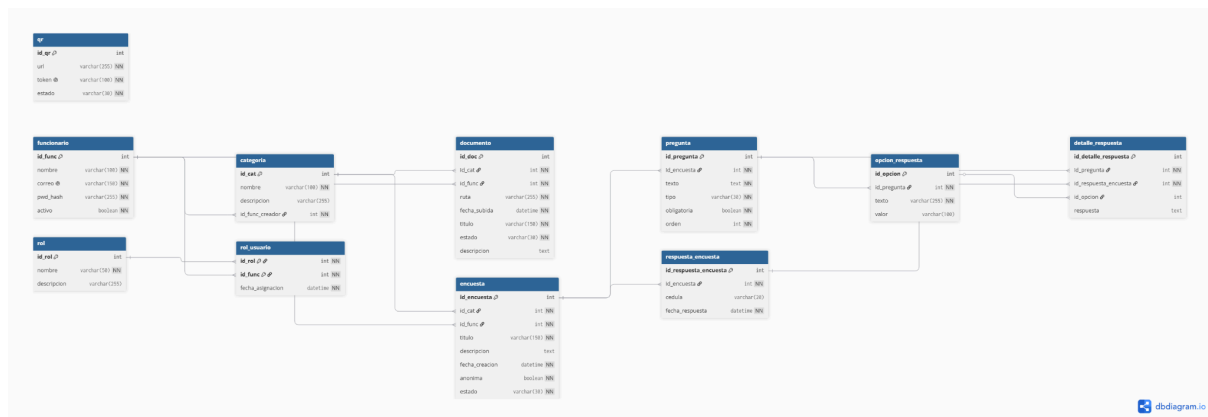
Se elaboraron tres representaciones principales. La primera corresponde al módulo de documentación, incluyendo las entidades relacionadas con funcionarios, roles, categorías, documentos, acceso mediante QR y encuestas integradas dentro de las categorías. La segunda corresponde al módulo de ambulancias, donde se representan los traslados, vehículos, estados, historial de cambios y funcionarios involucrados. Finalmente, se realizó un diagrama integrado que unifica ambos módulos mediante la entidad común Funcionario y la estructura de roles.

Esta representación visual permite comprender con mayor claridad cómo se organiza la información dentro del sistema y cómo se relacionan las distintas partes del proyecto. Además, sirve como base para el posterior pasaje al modelo relacional y para la creación de los scripts SQL correspondientes.

#### Diagrama del módulo de documentación

Este diagrama representa las entidades principales vinculadas a la gestión de documentación para pacientes. Incluye las tablas Funcionario, Rol, Rol\_Usuario, Categoría, Documento, QR, Encuesta, Pregunta, Opcion\_Respuesta, Respuesta\_Encuesta y Detalle\_Respuesta.

En este modelo, las encuestas se integran dentro de la estructura de categorías, permitiendo que una categoría pueda agrupar tanto documentos informativos como encuestas relacionadas con un servicio, área o actividad del hospital.



Enlace al diagrama:

<https://dbdiagram.io/d/Documentacon-6a39a8795c789b8acbdb158d>

## Diagrama del módulo de ambulancias

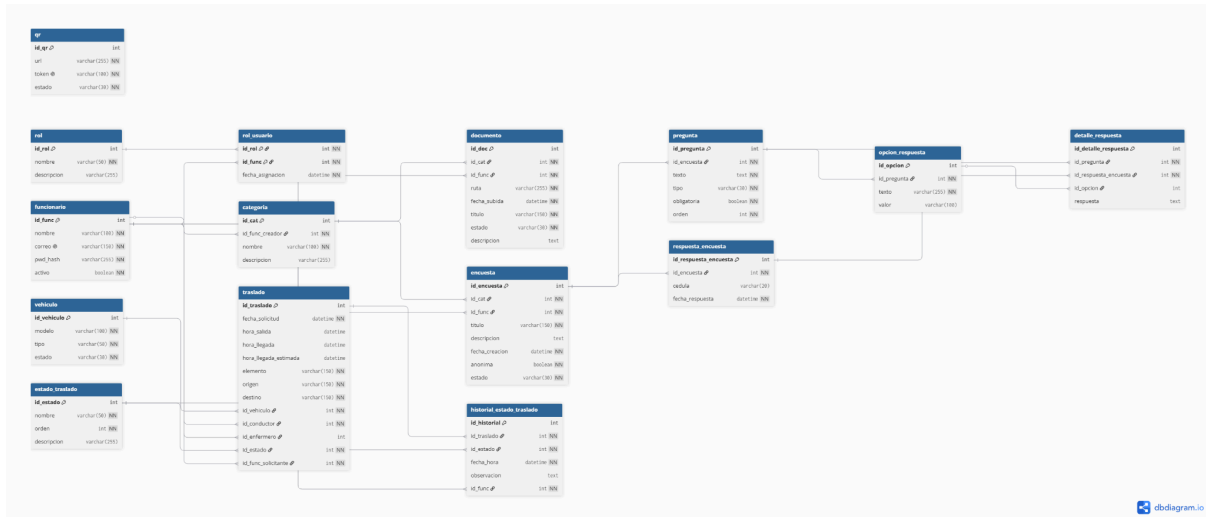
Este diagrama representa las entidades principales vinculadas a la gestión y seguimiento de traslados. Incluye las tablas Funcionario, Rol, Rol\_Usuario, Traslado, Vehículo, Estado\_Traslado e Historial\_Estado\_Traslado.

En este modelo, la entidad Funcionario se reutiliza para representar distintos actores internos del proceso, como el funcionario solicitante, el conductor, el enfermero y el usuario que registra cambios de estado.



Este diagrama integra los dos módulos principales del sistema S.I.G.S.M. en un único modelo. La integración se realiza principalmente mediante la entidad Funcionario y la relación con Rol y Rol\_Usuario, evitando duplicar usuarios en cada módulo.

S.I.G.S.M - I.S.B.O - Empre S.A



Enlace al diagrama: <https://dbdiagram.io/d/Unificado-6a39ad379340ecc065f1538f>

### 5.3 Restricciones no estructurales

Las restricciones no estructurales son reglas de negocio que no siempre pueden representarse directamente en el Diagrama Entidad-Relación, pero que deben cumplirse durante el funcionamiento del sistema. Estas restricciones permiten asegurar que los datos ingresados sean coherentes con la realidad del Hospital de Clínicas y con la lógica operativa de los módulos desarrollados.

Para documentarlas se especifica la descripción de la regla, las entidades o atributos involucrados y la forma en que se implementará dentro del sistema. La implementación podrá realizarse mediante validaciones en el frontend, validaciones en PHP, restricciones en la base de datos o una combinación de estos mecanismos.

#### 5.3.1 Restricciones del módulo de documentación

| Restricción                                                                                                                                                                        | Entidades involucradas                                                                  | Implementación                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Todo documento debe pertenecer a una categoría válida. Esto permite mantener organizada la documentación y evita que existan documentos sin clasificación.                         | Documento.id_cat,<br>Categoría.id_cat                                                   | Se implementará mediante una clave foránea en la base de datos y validación en PHP antes de guardar el documento.                                   |
| Todo documento debe estar asociado a un funcionario responsable de su carga o administración. Esto permite mantener trazabilidad sobre quién gestionó cada documento.              | Documento.id_func,<br>Funcionario.id_func                                               | Se implementará mediante clave foránea y validación en PHP, verificando que el funcionario exista y esté activo.                                    |
| No se permitirá cargar documentos sin título, ruta de archivo, fecha de subida o estado. Estos datos son mínimos para que el documento pueda gestionarse correctamente.            | Documento.titulo,<br>Documento.ruta,<br>Documento.fecha_s<br>ubida,<br>Documento.estado | Se validará en el frontend antes del envío y nuevamente en PHP. En la base de datos se definirán campos obligatorios.                               |
| Los documentos inactivos no deberán mostrarse a los pacientes. Podrán conservarse en la base de datos para fines administrativos, pero no estarán disponibles en la vista pública. | Documento.estado                                                                        | Se implementará mediante lógica en PHP, filtrando únicamente documentos con estado activo. También podrá reforzarse en consultas SQL.               |
| No deberán existir dos documentos activos con el mismo título dentro de una misma categoría. Esto evita confusiones al momento de consultar la información.                        | Documento.titulo,<br>Documento.id_cat,<br>Documento.estado                              | Se validará en PHP antes de registrar o actualizar un documento. También podrá implementarse mediante una restricción UNIQUE compuesta si se define |

|                                                                                                                                                                           |                                                                                      |                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                           |                                                                                      | como regla estricta.                                                                                                                          |
| Las categorías deberán tener nombre obligatorio. Una categoría sin nombre no permite organizar correctamente los documentos ni las encuestas asociadas.                   | Categoría.nombre                                                                     | Se validará en el frontend, en PHP y mediante campo obligatorio en la base de datos.                                                          |
| El acceso mediante QR deberá estar activo para permitir el ingreso al módulo de documentación. Si el QR se encuentra inactivo, no deberá permitir el acceso al contenido. | QR.estado,<br>QR.token, QR.url                                                       | Se implementará mediante validación en PHP al recibir el token o acceder a la URL. Solo se permitirá el acceso si el estado del QR es activo. |
| Toda encuesta debe pertenecer a una categoría válida. Esto permite integrarla dentro de la organización general del módulo de documentación.                              | Encuesta.id_cat,<br>Categoría.id_cat                                                 | Se implementará mediante clave foránea y validación en PHP.                                                                                   |
| Toda encuesta debe estar asociada a un funcionario responsable de su creación o administración.                                                                           | Encuesta.id_func,<br>Funcionario.id_func                                             | Se implementará mediante clave foránea y validación en PHP, verificando que el funcionario exista y esté activo.                              |
| Una encuesta debe tener título, fecha de creación, estado y definición de si es anónima o no. Estos datos son necesarios para su administración.                          | Encuesta.titulo,<br>Encuesta.fecha_creacion,<br>Encuesta.estado,<br>Encuesta.anonima | Se validará en el frontend y en PHP. En la base de datos se definirán campos obligatorios.                                                    |
| Una encuesta debe tener al menos una pregunta activa para poder ser publicada. No tiene sentido habilitar                                                                 | Encuesta.id_encuesta,<br>Pregunta.id_encuesta                                        | Se implementará mediante validación en PHP antes de cambiar el estado de la encuesta a activa o                                               |

|                                                                                                                                         |                                                                                  |                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| una encuesta vacía.                                                                                                                     |                                                                                  | publicada.                                                                                                                      |
| Las preguntas obligatorias deberán ser respondidas antes de enviar la encuesta.                                                         | Pregunta.obligatoria, Detalle_Respuesta.r espuesta, Detalle_Respuesta.i d_opcion | Se validará en el frontend para orientar al usuario y en PHP antes de guardar la respuesta.                                     |
| Las preguntas de tipo cerrado o selección deberán tener opciones de respuesta asociadas.                                                | Pregunta.tipo, Opcion_Respuesta.i d_pregunta                                     | Se validará en PHP antes de publicar la encuesta. También puede verificarse mediante consultas SQL.                             |
| Las respuestas de una encuesta deben estar asociadas a una encuesta existente.                                                          | Respuesta_Encuest a.id_encuesta, Encuesta.id_encues ta                           | Se implementará mediante clave foránea en la base de datos.                                                                     |
| Cada detalle de respuesta debe estar asociado a una respuesta de encuesta y a una pregunta válida.                                      | Detalle_Respuesta.i d_respuesta_encue sta, Detalle_Respuesta.i d_pregunta        | Se implementará mediante claves foráneas.                                                                                       |
| Si una encuesta es anónima, no se deberá almacenar la cédula del usuario que responde.                                                  | Encuesta.anonima, Respuesta_Encuest a.cedula                                     | Se implementará mediante validación en PHP. Si la encuesta tiene anonima = true, el campo cedula deberá guardarse vacío o nulo. |
| Si una pregunta es de tipo abierto, no será obligatorio registrar una opción de respuesta. En ese caso se utilizará el campo respuesta. | Pregunta.tipo, Detalle_Respuesta.i d_opcion, Detalle_Respuesta.r espuesta        | Se implementará mediante validación en PHP según el tipo de pregunta.                                                           |
| Si una pregunta es de tipo cerrado, la opción seleccionada deberá existir dentro de las opciones asociadas a esa pregunta.              | Pregunta.id_pregunt a, Opcion_Respuesta.i d_opcion, Detalle_Respuesta.i          | Se validará en PHP y se reforzará mediante claves foráneas.                                                                     |

|  |          |  |
|--|----------|--|
|  | d_opcion |  |
|--|----------|--|

### 5.3.2 Restricciones del módulo de ambulancias

| Restricción                                                                                                                               | Entidades involucradas                                                              | Implementación                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Todo traslado debe tener fecha de solicitud. Esta fecha permite registrar cuándo se generó la solicitud dentro del sistema.               | Traslado.fecha_solicitud                                                            | Se implementará mediante campo obligatorio en la base de datos y asignación automática desde PHP. |
| Todo traslado debe tener origen y destino. No se permitirá registrar una solicitud sin conocer desde dónde parte y hacia dónde se dirige. | Traslado.origen,<br>Traslado.destino                                                | Se validará en el frontend y en PHP. En la base de datos se definirán como campos obligatorios.   |
| Todo traslado debe tener un elemento o paciente a trasladar. Esto permite identificar qué se transporta.                                  | Traslado.elemento                                                                   | Se validará en el frontend, en PHP y mediante campo obligatorio en la base de datos.              |
| Todo traslado debe tener un conductor responsable.                                                                                        | Traslado.id_conductor,<br>Funcionario.id_func                                       | Se implementará mediante clave foránea y validación en PHP.                                       |
| El conductor asignado debe ser un funcionario activo y con rol habilitado para cumplir esa función.                                       | Traslado.id_conductor,<br>Funcionario.activo,<br>Rol.nombre,<br>Rol_Usuario.id_func | Se validará en PHP consultando la relación entre Funcionario, Rol y Rol_Usuario.                  |
| En los casos que corresponda, el traslado deberá tener un enfermero                                                                       | Traslado.id_enfermero,<br>Funcionario.id_func                                       | Se implementará mediante validación en PHP según el tipo de traslado o elemento                   |



| asignado.                                                                                                                           |                                                                                                                                                | trasladado.                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| El enfermero asignado debe ser un funcionario activo y con rol habilitado para cumplir esa función.                                 | Traslado.id_enfermero,<br>Funcionario.activo,<br>Rol.nombre,<br>Rol_Usuario.id_func                                                            | Se validará en PHP consultando los roles asignados al funcionario.                                                                 |
| Todo traslado debe tener un funcionario solicitante. Esto permite identificar quién generó la solicitud dentro del sistema.         | Traslado.id_func_solicitante,<br>Funcionario.id_func                                                                                           | Se implementará mediante clave foránea y validación en PHP, tomando el usuario autenticado.                                        |
| Todo traslado debe tener un estado actual válido.                                                                                   | Traslado.id_estado,<br>Estado_Traslado.id_estado                                                                                               | Se implementará mediante clave foránea en la base de datos.                                                                        |
| La hora de llegada real no puede ser anterior a la hora de salida.                                                                  | Traslado.hora_salida,<br>Traslado.hora_llegada                                                                                                 | Se validará en el frontend y en PHP. También podría implementarse mediante una restricción CHECK en la base de datos.              |
| La hora de llegada estimada no debería ser anterior a la hora de salida.                                                            | Traslado.hora_salida,<br>Traslado.hora_llegada_estimada                                                                                        | Se validará en el frontend y en PHP antes de registrar el traslado.                                                                |
| Un traslado no debería cambiar a un estado anterior dentro del flujo normal de trabajo, salvo corrección administrativa autorizada. | Traslado.id_estado,<br>Estado_Traslado.orden                                                                                                   | Se implementará mediante validación en PHP, comparando el orden del estado actual con el nuevo estado solicitado.                  |
| Cada cambio de estado debe quedar registrado en el historial del traslado.                                                          | Historial_Estado_Traslado.id_traslado,<br>Historial_Estado_Traslado.id_estado,<br>Historial_Estado_Traslado.fecha_hora,<br>Historial_Estado_Tr | Se implementará mediante lógica en PHP al actualizar el estado. También podría reforzarse mediante un TRIGGER en la base de datos. |

|                                                                                                                                             |                                                              |                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|                                                                                                                                             | aslado.id_func                                               |                                                                                               |
| Un vehículo fuera de servicio no deberá poder asignarse a un traslado.                                                                      | Vehículo.estado,<br>Traslado.id_vehiculo                     | Se validará en PHP antes de crear o actualizar el traslado.                                   |
| El tipo de vehículo debe ser compatible con el tipo de elemento trasladado. No todos los traslados pueden realizarse en cualquier vehículo. | Vehículo.tipo,<br>Traslado.elemento                          | Se implementará mediante validación en PHP según reglas definidas para cada tipo de traslado. |
| El funcionario que registra un cambio de estado debe existir y estar activo.                                                                | Historial_Estado_Tr<br>aslado.id_func,<br>Funcionario.activo | Se implementará mediante clave foránea y validación en PHP.                                   |

### 5.3.3 Forma de implementación de las restricciones

Las restricciones no estructurales del sistema se implementarán mediante una combinación de validaciones en el frontend, validaciones en el backend y restricciones en la base de datos.

En el frontend, desarrollado con React, se realizarán validaciones iniciales para mejorar la experiencia de usuario. Por ejemplo, se indicarán campos obligatorios, se evitará el envío de formularios incompletos y se mostrarán mensajes de error cuando los datos ingresados no sean válidos. Estas validaciones ayudan al usuario, pero no son suficientes por sí solas, ya que pueden ser omitidas o manipuladas.

En el backend, desarrollado con PHP, se realizarán las validaciones principales antes de guardar o modificar información. PHP verificará que los datos recibidos sean coherentes, que las entidades relacionadas existan, que los funcionarios

tengan los roles correspondientes y que se respeten las reglas de negocio definidas para documentación, encuestas y traslados.

En la base de datos se utilizarán claves primarias y claves foráneas para mantener la integridad referencial entre las tablas. También se podrán utilizar campos obligatorios, restricciones de unicidad y, cuando corresponda, restricciones CHECK o TRIGGER para reforzar determinadas reglas.

Por ejemplo, la relación entre Documento y Categoría se garantizará mediante una clave foránea, mientras que la regla de que un documento tenga título y ruta se reforzará mediante campos obligatorios. En el caso de las encuestas anónimas, PHP deberá verificar que no se almacene la cédula cuando el atributo anonima sea verdadero. En el caso de ambulancias, PHP deberá validar que el conductor y el enfermero asignados sean funcionarios activos y tengan los roles adecuados.

De esta forma, el sistema no dependerá de un único mecanismo de validación, sino que combinará distintas capas de control para reducir errores, mantener la coherencia de los datos y asegurar un funcionamiento más confiable.

## 5.4 Esquema relacional normalizado

El esquema relacional normalizado surge a partir de la transformación del Diagrama Entidad-Relación en un conjunto de tablas SQL. En esta etapa, cada entidad identificada en el modelo conceptual se convierte en una tabla, sus atributos pasan a ser columnas y las relaciones entre entidades se representan mediante claves primarias y claves foráneas.

El objetivo de este proceso es obtener una estructura de base de datos clara, consistente y preparada para ser implementada en MySQL/MariaDB. Para ello, se

identifican las tablas principales, sus columnas, tipos de datos, claves primarias, claves foráneas y las decisiones de diseño tomadas durante el modelado.

#### **5.4.1 Pasaje del DER al modelo relacional**

A partir del DER inicial se realiza el pasaje al modelo relacional. Las entidades principales del sistema se transforman en tablas independientes. Las relaciones uno a muchos se implementan incorporando la clave primaria de la tabla del lado “uno” como clave foránea en la tabla del lado “muchos”. Las relaciones muchos a muchos se resuelven mediante tablas intermedias.

En el sistema S.I.G.S.M., la entidad Funcionario funciona como entidad común para ambos módulos. Esto permite reutilizar la misma estructura de usuarios internos tanto en documentación y encuestas como en ambulancias. La relación entre Funcionario y Rol se resuelve mediante la tabla Rol\_Usuario, ya que un funcionario puede tener varios roles y un rol puede estar asignado a varios funcionarios.

En el módulo de documentación, las tablas principales son Categoría, Documento, QR, Encuesta, Pregunta, Opcion\_Respuesta, Respuesta\_Encuesta y Detalle\_Respuesta. Las encuestas se integran dentro de la estructura de categorías, por lo que cada encuesta pertenece a una categoría determinada.

En el módulo de ambulancias, las tablas principales son Vehículo, Estado\_Traslado, Traslado e Historial\_Estado\_Traslado. La tabla Traslado se vincula con Funcionario para representar al conductor, enfermero y funcionario solicitante. También se vincula con Vehículo y Estado\_Traslado para controlar el vehículo asignado y el estado actual del traslado.

A continuación se presenta el esquema relacional propuesto.

| 5.4.2 Tabla funcionario |              |        |                                                             |
|-------------------------|--------------|--------|-------------------------------------------------------------|
| Columna                 | Tipo de dato | Clave  | Descripción                                                 |
| id_func                 | INT          | PK     | Identificador único del funcionario.                        |
| nombre                  | VARCHAR(100) |        | Nombre del funcionario.                                     |
| correo                  | VARCHAR(150) | UNIQUE | Correo institucional o de acceso.                           |
| pwd_hash                | VARCHAR(255) |        | Contraseña almacenada de forma cifrada o hasheada.          |
| activo                  | BOOLEAN      |        | Indica si el funcionario se encuentra activo en el sistema. |

| 5.4.3 Tabla rol |              |       |                                          |
|-----------------|--------------|-------|------------------------------------------|
| Columna         | Tipo de dato | Clave | Descripción                              |
| id_rol          | INT          | PK    | Identificador único del rol.             |
| nombre          | VARCHAR(100) |       | Nombre del rol.                          |
| descripcion     | VARCHAR(255) |       | Descripción del rol y permisos generales |

| 5.4.4 Tabla rol_usuario |              |         |                                               |
|-------------------------|--------------|---------|-----------------------------------------------|
| Columna                 | Tipo de dato | Clave   | Descripción                                   |
| id_rol                  | INT          | PK / FK | Referencia al rol asignado.                   |
| id_func                 | INT          | PK / FK | Referencia al funcionario.                    |
| fecha_asignacion        | DATETIME     |         | Fecha en que se asignó el rol al funcionario. |

| 5.4.5 Tabla categoria |              |       |                                                 |
|-----------------------|--------------|-------|-------------------------------------------------|
| Columna               | Tipo de dato | Clave | Descripción                                     |
| id_cat                | INT          | PK    | Identificador de la categoría.                  |
| id_func               | INT          | FK    | Funcionario que creó o administra la categoría. |
| nombre                | VARCHAR(100) |       | Nombre de la categoría                          |
| descripcion           | VARCHAR(255) |       | Descripción de la categoría                     |

| 5.4.6 Tabla documento |              |       |                                           |
|-----------------------|--------------|-------|-------------------------------------------|
| Columna               | Tipo de dato | Clave | Descripción                               |
| id_doc                | INT          | PK    | Identificador del documento.              |
| id_func               | INT          | FK    | Funcionario que administra el documento.  |
| titulo                | VARCHAR(100) |       | Título del documento                      |
| descripcion           | TEXT         |       | Descripción del documento                 |
| estado                | VARCHAR(30)  |       | Estado del documento                      |
| ruta                  | VARCHAR(255) |       | Ubicación del archivo en el servidor      |
| id_cat                | INT          | FK    | Categoría a la que pertenece el documento |
| fecha-subida          | DATETIME     |       | Fecha de subida el archivo                |

| 5.4.7 Tabla QR |              |       |                       |
|----------------|--------------|-------|-----------------------|
| Columna        | Tipo de dato | Clave | Descripción           |
| id_qr          | INT          | PK    | Identificador del QR. |

| 5.4.7 Tabla QR |              |        |                                     |
|----------------|--------------|--------|-------------------------------------|
| url            | VARCHAR(255) |        | Dirección a la que apunta el código |
| token          | VARCHAR(100) | UNIQUE | Token asociado al acceso            |
| estado         | VARCHAR(30)  |        | Estado del código                   |

La tabla QR se mantiene como una tabla de acceso general al módulo de documentación, ya que en esta etapa se plantea un único QR para acceder a la aplicación o sección correspondiente, y no un QR independiente por cada documento.

| 5.4.8 Tabla encuesta |              |       |                                  |
|----------------------|--------------|-------|----------------------------------|
| Columna              | Tipo de dato | Clave | Descripción                      |
| id_encuesta          | INT          | PK    | Identificador de la encuesta.    |
| id_cat               | INT          | FK    | Categoría a la que pertenece     |
| id_func              | INT          | FK    | Funcionario que crea la encuesta |
| titulo               | VARCHAR(150) |       | Título de la encuesta            |
| descripcion          | TEXT         |       | Descripción de la encuesta       |
| fecha_creacion       | DATETIME     |       | Fecha de creación de la encuesta |



| 5.4.8 Tabla encuesta |             |  |                       |
|----------------------|-------------|--|-----------------------|
| anonima              | BOOLEAN     |  | Indica si es anónima  |
| estado               | VARCHAR(30) |  | Estado de la encuesta |

| 5.4.9 Tabla pregunta |              |       |                                                 |
|----------------------|--------------|-------|-------------------------------------------------|
| Columna              | Tipo de dato | Clave | Descripción                                     |
| id_pregunta          | INT          | PK    | Identificador de la pregunta.                   |
| id_encuesta          | INT          | FK    | Identifica la encuesta a la que pertenece       |
| texto                | TEXT         |       | Texto de la pregunta                            |
| tipo                 | VARCHAR(30)  |       | Tipo de pregunta (abierta, cerrada o selección) |
| obligatoria          | BOOLEAN      |       | Indica si es obligatoria                        |
| orden                | INT          |       | Orden de aparición de la pregunta               |

| 5.4.10 Tabla opcion_respuesta |              |       |                             |
|-------------------------------|--------------|-------|-----------------------------|
| Columna                       | Tipo de dato | Clave | Descripción                 |
| id_opcion                     | INT          | PK    | Identificador de la opcion. |
| id_pregunta                   | INT          | FK    | Pregunta a la que pertenece |
| texto                         | VARCHAR(255) |       | Texto de la opción          |
| valor                         | VARCHAR(100) |       | Valor interno de la opción  |

| 5.4.11 Tabla respuesta_encuesta |              |       |                                                 |
|---------------------------------|--------------|-------|-------------------------------------------------|
| Columna                         | Tipo de dato | Clave | Descripción                                     |
| id_respuesta_encuesta           | INT          | PK    | Identificador de la respuesta enviada           |
| id_encuesta                     | INT          | FK    | Encuesta respondida                             |
| cedula                          | VARCHAR(20)  | NULL  | Cedula del usuario si la encuesta no es anonima |
| fecha_respuesta                 | DATETIME     |       | Fecha de la respuesta                           |

#### 5.4.12 Tabla detalle\_respuesta

| Columna               | Tipo de dato | Clave     | Descripción                                        |
|-----------------------|--------------|-----------|----------------------------------------------------|
| id_detalle_respuesta  | INT          | PK        | Identifica la respuesta del usuario a una pregunta |
| id_pregunta           | INT          | FK        | Identifica la pregunta                             |
| id_respuesta_encuesta | INT          | FK        | Respuesta general a la que pertenece               |
| id_opcion             | INT          | FK / NULL | Opción seleccionada si corresponde                 |
| respuesta             | TEXT         | NULL      | Respuesta escrita por el usuario                   |

#### 5.4.13 Tabla vehículo

| Columna     | Tipo de dato | Clave | Descripción                 |
|-------------|--------------|-------|-----------------------------|
| id_vehiculo | INT          | PK    | Identificador del vehículo. |
| modelo      | VARCHAR(100) |       | Modelo del vehículo         |
| tipo        | VARCHAR(30)  |       | Tipo de vehículo            |
| estado      | VARCHAR(30)  |       | Estado del vehículo         |

| 5.4.14 Tabla estado_traslado |              |       |                                        |
|------------------------------|--------------|-------|----------------------------------------|
| Columna                      | Tipo de dato | Clave | Descripción                            |
| id_estado                    | INT          | PK    | Identificador del estado del traslado. |
| nombre                       | VARCHAR(50)  |       | Nombre del estado                      |
| orden                        | INT          |       | Orden lógico                           |
| descripcion                  | VARCHAR(255) |       | Descripción del estado                 |

| 5.4.15 Tabla traslado |              |       |                                       |
|-----------------------|--------------|-------|---------------------------------------|
| Columna               | Tipo de dato | Clave | Descripción                           |
| id_traslado           | INT          | PK    | Identificador del traslado.           |
| fecha_solicitud       | DATETIME     |       | Fecha en la que se generó el traslado |
| hora_salida           | DATETIME     | NULL  | Hora de salida del traslado           |
| hora_llegaad          | DATETIME     | NULL  | Hora real de llegada                  |
| hora_llegada_estimada | DATETIME     | NULL  | Hora estimada de llegada              |
| elemento              | VARCHAR(150) |       | Paciente o elemento                   |

| 5.4.15 Tabla traslado |              |    |                                      |
|-----------------------|--------------|----|--------------------------------------|
|                       |              |    | trasladado                           |
| origen                | VARCHAR(150) |    | Origen del traslado                  |
| destino               | VARCHAR(150) |    | Destino del traslado                 |
| id_vehiculo           | INT          | FK | Vehículo asignado                    |
| id_conductor          | INT          | FK | Conductor asignado                   |
| id_enfermero          | INT          | FK | Enfermero asignado                   |
| id_estado             | INT          | FK | Estado actual del traslado           |
| id_func_solicitante   | INT          | FK | Funcionario que solicita el traslado |

| 5.4.16 Tabla historial_estado_traslado |              |       |                                               |
|----------------------------------------|--------------|-------|-----------------------------------------------|
| Columna                                | Tipo de dato | Clave | Descripción                                   |
| id_historial                           | INT          | PK    | Identificador del registro histórico.         |
| id_traslado                            | INT          | FK    | Traslado al que pertenece el cambio de estado |
| id_estado                              | INT          | FK    | Estado registrado en el historial             |
| fecha_hora                             | DATETIME     |       | Fecha y hora del                              |

| 5.4.16 Tabla historial_estado_traslado |      |      |                                    |
|----------------------------------------|------|------|------------------------------------|
|                                        |      |      | cambio de estado                   |
| observacion                            | TEXT | NULL | Observación asociada               |
| id_func                                | INT  | FK   | Funcionario que registró el cambio |

#### 5.4.17 Identificación de claves primarias

Las claves primarias permiten identificar de forma única cada registro dentro de una tabla.

Las claves primarias propuestas son las siguientes:

- funcionario: id\_func
- rol: id\_rol
- rol\_usuario: clave compuesta por id\_rol e id\_func
- categoria: id\_cat
- documento: id\_doc
- qr: id\_qr
- encuesta: id\_encuesta
- pregunta: id\_pregunta
- opcion\_respuesta: id\_opcion
- respuesta\_encuesta: id\_respuesta\_encuesta
- detalle\_respuesta: id\_detalle\_respuesta
- vehiculo: id\_vehiculo
- estado\_traslado: id\_estado



- traslado: id\_traslado
- historial\_estado\_traslado: id\_historial

Cada clave primaria permite diferenciar registros dentro de su tabla y evita duplicaciones de identidad.

#### 5.4.18 Identificación de claves foráneas

Las claves foráneas permiten relacionar las tablas entre sí y mantener la integridad referencial del sistema.

Las claves foráneas propuestas son las siguientes:

- rol\_usuario.id\_rol referencia a rol.id\_rol
- rol\_usuario.id\_func referencia a funcionario.id\_func
- categoria.id\_func\_creador referencia a funcionario.id\_func
- documento.id\_cat referencia a categoria.id\_cat
- documento.id\_func referencia a funcionario.id\_func
- encuesta.id\_cat referencia a categoria.id\_cat
- encuesta.id\_func referencia a funcionario.id\_func
- pregunta.id\_encuesta referencia a encuesta.id\_encuesta
- opcion\_respuesta.id\_pregunta referencia a pregunta.id\_pregunta
- respuesta\_encuesta.id\_encuesta referencia a encuesta.id\_encuesta
- detalle\_respuesta.id\_pregunta referencia a pregunta.id\_pregunta
- detalle\_respuesta.id\_respuesta\_encuesta referencia a respuesta\_encuesta.id\_respuesta\_encuesta
- detalle\_respuesta.id\_opcion referencia a opcion\_respuesta.id\_opcion
- traslado.id\_vehiculo referencia a vehiculo.id\_vehiculo

- traslado.id\_conductor referencia a funcionario.id\_func
- traslado.id\_enfermero referencia a funcionario.id\_func
- traslado.id\_estado referencia a estado\_traslado.id\_estado
- traslado.id\_func\_solicitante referencia a funcionario.id\_func
- historial\_estado\_traslado.id\_traslado referencia a traslado.id\_traslado
- historial\_estado\_traslado.id\_estado referencia a estado\_traslado.id\_estado
- historial\_estado\_traslado.id\_func referencia a funcionario.id\_func

Estas relaciones permiten evitar registros huérfanos. Por ejemplo, no podrá existir un documento asociado a una categoría inexistente, una encuesta sin categoría, una pregunta sin encuesta o un traslado con un conductor que no exista como funcionario.

#### **5.4.19 Normalización hasta tercera forma normal**

El esquema propuesto fue revisado siguiendo el proceso de normalización hasta la Tercera Forma Normal, con el objetivo de reducir redundancias y evitar dependencias incorrectas entre los datos.

##### **Primera Forma Normal**

La Primera Forma Normal establece que cada celda debe contener un único valor atómico y que no deben existir listas ni grupos repetitivos dentro de un campo.

En el modelo propuesto, cada atributo almacena un único valor. Por ejemplo, en la tabla documento se separan los datos en columnas individuales como titulo, ruta, fecha\_subida, estado y descripcion. Del mismo modo, en traslado se separan fecha\_solicitud, hora\_salida, hora\_llegada, origen, destino y elemento.



En el caso de las encuestas, no se almacenan todas las preguntas ni respuestas dentro de una única columna. En su lugar, se crean tablas separadas para encuesta, pregunta, opcion\_respuesta, respuesta\_encuesta y detalle\_respuesta. Esta decisión permite evitar listas internas y representar correctamente la estructura del formulario.

### **Segunda Forma Normal**

La Segunda Forma Normal establece que todos los atributos deben depender completamente de la clave primaria y no solo de una parte de ella. Esta regla se aplica especialmente en tablas con claves compuestas.

La mayoría de las tablas del modelo utilizan claves primarias simples, por lo que sus atributos dependen directamente del identificador principal. Por ejemplo, en vehiculo, los atributos modelo, tipo y estado dependen de id\_vehiculo. En estado\_traslado, los atributos nombre, orden y descripcion dependen de id\_estado.

La tabla rol\_usuario utiliza una clave compuesta formada por id\_rol e id\_func. El atributo fecha\_asignacion depende de la combinación de ambos campos, ya que representa cuándo un rol específico fue asignado a un funcionario específico. Por lo tanto, la tabla cumple con Segunda Forma Normal.

### **Tercera Forma Normal**

La Tercera Forma Normal establece que los atributos que no son clave no deben depender de otros atributos que tampoco sean clave. Es decir, se deben evitar dependencias transitivas.

Para cumplir con esta forma normal, se separaron entidades que podrían haberse almacenado como texto repetido dentro de otras tablas. Por ejemplo, los roles no se guardan directamente como texto dentro de funcionario, sino en una tabla rol relacionada mediante rol\_usuario. Esto evita repetir nombres de roles y permite administrar los permisos de forma más ordenada.

También se separó estado\_traslado en una tabla propia. En lugar de guardar el nombre del estado directamente dentro de cada traslado, se almacena id\_estado como clave foránea. Esto evita inconsistencias como escribir un mismo estado de distintas formas.

Las categorías también se separan de los documentos y encuestas. En lugar de escribir el nombre de la categoría en cada documento o encuesta, se utiliza id\_cat como clave foránea. Esto permite mantener una única definición de cada categoría.

En el módulo de encuestas, las opciones de respuesta se separan de las preguntas y las respuestas se separan de los detalles de respuesta. Esta estructura evita repetir información y permite que una encuesta pueda tener múltiples preguntas, opciones y respuestas de forma ordenada.

En el módulo de ambulancias, la entidad funcionario se reutiliza para conductor, enfermero y solicitante, evitando crear tablas separadas con la misma información personal. Esta decisión reduce duplicación y permite centralizar la administración de usuarios internos.

Por lo tanto, el modelo propuesto cumple con Tercera Forma Normal en esta etapa inicial del proyecto.

#### **5.4.20 Modificaciones realizadas durante la normalización**

Durante el proceso de normalización se realizaron algunas modificaciones sobre el diseño inicial para mejorar la estructura de la base de datos.

En primer lugar, se decidió separar los roles de los funcionarios mediante las tablas rol y rol\_usuario. Una alternativa inicial podría haber sido almacenar el rol directamente dentro de la tabla funcionario, pero esto no permitiría asignar más de un rol a un mismo funcionario. Al usar rol\_usuario, el sistema permite que un funcionario pueda tener distintos permisos según las tareas que realice.

También se separaron las encuestas en varias tablas: encuesta, pregunta, opcion\_respuesta, respuesta\_encuesta y detalle\_respuesta. Una alternativa más simple hubiera sido almacenar toda la estructura de la encuesta en una sola tabla, pero eso generaría campos repetidos, poca flexibilidad y dificultad para analizar respuestas. La separación permite representar encuestas de forma más ordenada y extensible.

Se decidió vincular encuesta con categoria mediante id\_cat. Esta modificación permite que las encuestas formen parte de la misma organización general que los documentos, respetando la idea de que las encuestas son una sección integrada dentro del módulo de documentación.

En el módulo de ambulancias se incorporó id\_vehiculo dentro de traslado para representar qué vehículo se asigna a cada solicitud. Sin este campo, la entidad vehiculo quedaría desconectada del traslado y no sería posible registrar correctamente qué vehículo realizó cada operación.

También se incorporó `id_estado` dentro de `historial_estado_traslado`. Esta decisión permite registrar a qué estado cambió el traslado en cada momento. Sin este campo, el historial indicaría que ocurrió un cambio, pero no permitiría saber cuál fue el estado registrado.

#### **5.4.21 Justificación de decisiones de diseño**

Durante el diseño de la base de datos se tomaron distintas decisiones con el objetivo de mantener una estructura clara, flexible y coherente con los requerimientos del sistema.

Una decisión importante fue utilizar `Funcionario` como entidad común para ambos módulos. Esto permite evitar la duplicación de usuarios y centralizar la información de quienes acceden al sistema. De esta forma, un mismo funcionario puede cargar documentos, crear encuestas, solicitar traslados, actuar como conductor o registrar cambios de estado, dependiendo de los roles que tenga asignados.

También se decidió implementar la relación entre `Funcionario` y `Rol` mediante la tabla intermedia `Rol_Usuario`. La alternativa descartada fue almacenar un único rol directamente en la tabla `Funcionario`. Esa opción hubiera sido más simple, pero menos flexible, ya que impediría que una misma persona tuviera más de una función dentro del sistema.

Otra decisión fue integrar las encuestas dentro de las categorías del módulo de documentación. Esto permite que las encuestas no queden como un bloque aislado, sino organizadas bajo la misma estructura de clasificación que los documentos. La alternativa descartada fue modelar encuestas como un módulo completamente independiente, lo cual hubiera generado una separación innecesaria dentro del sistema.

En cuanto al QR, se decidió representarlo como un acceso general al módulo de documentación. Esto responde al criterio de que el QR no estará asociado a un documento individual, sino que funcionará como vía de ingreso a la aplicación o a la sección correspondiente. La alternativa descartada fue crear una relación directa entre QR y Documento, ya que eso representaría una lógica de “un QR por documento” que no corresponde con el enfoque actual del proyecto.

En el módulo de ambulancias se decidió que los campos `id_conductor`, `id_enfermero` e `id_func_solicitante` referencien a la tabla `Funcionario`. Esta decisión permite reutilizar la misma entidad para distintos actores del proceso. La alternativa descartada fue crear tablas separadas para conductor, enfermero y solicitante, lo cual duplicaría información y dificultaría el mantenimiento.

También se decidió utilizar una tabla `Historial_Estado_Traslado` para registrar los cambios de estado. La alternativa más simple hubiera sido guardar únicamente el estado actual en la tabla `Traslado`, pero eso impediría conocer la evolución del recorrido. Con el historial se conserva la trazabilidad del traslado durante todo su ciclo de vida.

Finalmente, se decidió no almacenar información clínica completa del paciente dentro de la base de datos local. En caso de necesitar identificar un paciente, se utilizará un dato mínimo como la cédula o un identificador externo, y la información completa podrá obtenerse desde una API o sistema externo autorizado. Esta decisión evita duplicar datos sensibles, reduce inconsistencias y mantiene el sistema más alineado con el principio de almacenar solo la información necesaria.

## **6. Configuración del entorno de desarrollo**

### **6.1 Objetivo del entorno de desarrollo**

El objetivo de esta sección es documentar la configuración del entorno de desarrollo utilizado para el proyecto S.I.G.S.M., de forma que cualquier integrante del equipo o docente pueda replicar las condiciones necesarias para ejecutar y probar el sistema desde cero.

El entorno de desarrollo debe permitir trabajar con los dos módulos principales del sistema: el módulo de gestión y acceso digital a documentación para pacientes, y el módulo de gestión y seguimiento de transporte de ambulancias. Para ello, se requiere contar con herramientas para ejecutar el backend, gestionar la base de datos, desarrollar el frontend, administrar dependencias, trabajar con control de versiones y probar la aplicación desde un navegador.

La configuración propuesta busca mantener una separación clara entre las distintas partes del sistema. Por un lado, el backend será desarrollado en PHP y ejecutado mediante Apache, utilizando XAMPP como entorno local. Por otro lado, el frontend será desarrollado con React, Material UI y Vite, utilizando Node.js y npm para instalar dependencias y levantar el servidor de desarrollo. La base de datos será gestionada mediante MySQL/MariaDB, permitiendo almacenar la información relacionada con documentos, encuestas, usuarios, traslados, vehículos y estados.

Además, se utilizará Visual Studio Code como entorno principal de desarrollo y Git/GitHub como herramientas de control de versiones. Esta configuración permitirá trabajar de forma ordenada, documentada y colaborativa durante todo el desarrollo del proyecto.



## **6.2 Requisitos de software**

Para poder ejecutar correctamente el entorno de desarrollo del proyecto S.I.G.S.M., será necesario contar con las siguientes herramientas instaladas y configuradas.

### **6.2.1 Sistema operativo**

El desarrollo podrá realizarse desde un sistema operativo de escritorio como Windows o GNU/Linux. En el caso del servidor de pruebas, se utilizará una máquina virtual con GNU/Linux, buscando mantener compatibilidad con la infraestructura prevista para el despliegue del sistema.

El uso de una máquina virtual permite simular un entorno más cercano al servidor final, probar configuraciones de red, instalar paquetes necesarios y verificar el comportamiento de la aplicación en un sistema similar al que podría utilizarse en producción.

Sistema operativo utilizado en desarrollo: Windows 11

Sistema operativo utilizado en la máquina virtual: Fedora Server 43

### **6.2.2 XAMPP 8.2.12 / Apache**

Se utilizará XAMPP 8.2.12 como entorno local para ejecutar Apache, PHP y MySQL/MariaDB. Esta herramienta facilita la instalación y configuración inicial del backend, ya que integra en un mismo paquete los servicios necesarios para trabajar con aplicaciones web desarrolladas en PHP.



Apache será utilizado como servidor web para servir los archivos del backend y permitir la ejecución de scripts PHP. Durante el desarrollo, los archivos del backend podrán ubicarse dentro del directorio correspondiente de XAMPP, generalmente llamado htdocs.

El enlace de descarga es el siguiente: [XAMPP Installers and Downloads for Apache Friends](#)

### **6.2.3 PHP**

PHP será utilizado como lenguaje backend del sistema. Se encargará de procesar solicitudes, validar información, conectarse con la base de datos y gestionar la lógica interna de los módulos.

La versión de PHP utilizada será la incluida en XAMPP 8.2.12. Para verificar la versión instalada, se puede ejecutar el siguiente comando en la terminal:

```
php -v
```

En caso de que el comando no sea reconocido, será necesario revisar que PHP esté agregado a las variables de entorno del sistema o ejecutar PHP desde la ruta donde se encuentra instalado dentro de XAMPP.

### **6.2.4 MariaDB/MySQL**

MySQL/MariaDB será utilizado como sistema de gestión de base de datos relacional. Permitirá almacenar la información del sistema de forma estructurada,





incluyendo documentos, categorías, encuestas, respuestas, usuarios, traslados, vehículos, conductores, enfermeros y estados de seguimiento.

El acceso a la base de datos podrá realizarse mediante phpMyAdmin, incluido en XAMPP. Para verificar su funcionamiento, se debe iniciar Apache y MySQL/MariaDB desde XAMPP y abrir en el navegador: <http://localhost/phpmyadmin>

Desde phpMyAdmin se podrá crear la base de datos del proyecto, importar scripts SQL, consultar tablas y verificar registros.

Versión de MariaDB utilizada: 10.4.32

### **6.2.5 Node.js y npm**

Node.js será necesario para trabajar con el frontend desarrollado en React mediante Vite. Aunque Node.js no será utilizado como backend principal del sistema, sí será indispensable durante el desarrollo del frontend.

Mediante Node.js y npm se podrán instalar las dependencias necesarias, ejecutar scripts del proyecto, levantar el servidor local de desarrollo y generar la versión final optimizada del frontend.

Para verificar que Node.js y npm están instalados correctamente, se deben ejecutar los siguientes comandos:

```
node -v
```

```
npm -v
```



Si ambos comandos muestran una versión instalada, significa que Node.js y npm están correctamente disponibles en el sistema.

De no ser así se puede descargar desde el siguiente enlace: [Node.js — Descarga Node.js®](#)

Versión de Node utilizada: 22.18.0

Versión de NPM utilizada: 11.5.2

### 6.2.6 Git

Git será utilizado como sistema de control de versiones. Permitirá registrar los cambios realizados en el proyecto, trabajar con ramas, documentar avances y recuperar versiones anteriores en caso de errores.

Para verificar que Git está instalado correctamente, se debe ejecutar:

```
git --version
```

También será necesario configurar el nombre de usuario y correo electrónico del integrante que realizará commits:

```
git config --global user.name "Nombre del integrante"
```

```
git config --global user.email "correo@ejemplo.com"
```

Estos datos permitirán identificar correctamente los cambios realizados por cada integrante del equipo.



Versión utilizada: 2.43.0

Se puede descargar la herramienta a través de el siguiente enlace: [Git - Install](#)

### **6.2.7 Visual Studio Code**

Visual Studio Code será utilizado como entorno principal de desarrollo. Permite trabajar en un mismo espacio con archivos PHP, JavaScript, React, CSS, configuración de Vite, scripts SQL y control de versiones mediante Git.

Su uso facilita la organización del proyecto, la edición de código, el autocompletado, el formateo automático y la integración con extensiones útiles para el desarrollo.

Versión de Visual Studio Code utilizada: 1.125.1

Se puede obtener desde el siguiente enlace: [Visual Studio Code - The open source AI code editor | Your home for multi-agent development](#)

### **6.2.8 Navegador web actualizado**

Será necesario contar con un navegador web actualizado para probar las interfaces del sistema. El navegador permitirá verificar el funcionamiento del frontend, comprobar la responsividad de las pantallas y acceder al backend servido por Apache.

Navegadores recomendados: Google Chrome, Microsoft Edge, Mozilla Firefox o similar.



### **6.2.9 VirtualBox o VMware para servidor de pruebas**

Para simular un entorno de servidor, se podrá utilizar VirtualBox o VMware Workstation. Estas herramientas permiten crear una máquina virtual con GNU/Linux, instalar los paquetes necesarios y probar el sistema en un entorno más cercano al de producción.

La máquina virtual podrá utilizarse para instalar XAMPP o los servicios equivalentes necesarios para ejecutar Apache, PHP, MySQL/MariaDB y otras herramientas relacionadas con el despliegue.

Herramienta de virtualización utilizada: VirtualBox

Requisitos mínimos recomendados para la máquina virtual:

Procesador: 2 núcleos

Memoria RAM: 4 GB

Disco duro: 50 GB

Red: Adaptador NAT o adaptador puente

Sistema operativo: Fedora Server 43

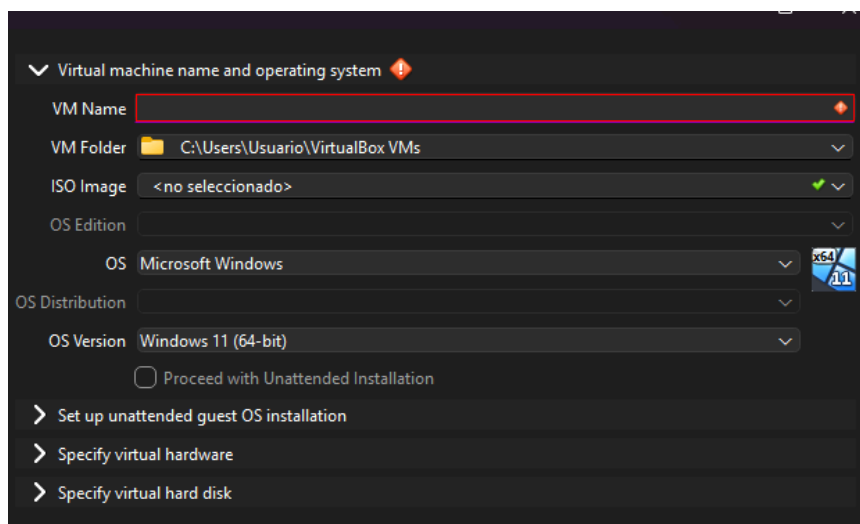
Estos valores permiten ejecutar correctamente un entorno inicial compuesto por servidor web, PHP, base de datos y herramientas de administración del sistema. Para un entorno de producción deberán considerarse otros factores, como cantidad de usuarios, volumen de documentos, cantidad de registros y uso de la base de datos.

Se puede obtener VirtualBox desde el siguiente enlace: [Oracle VirtualBox](#)

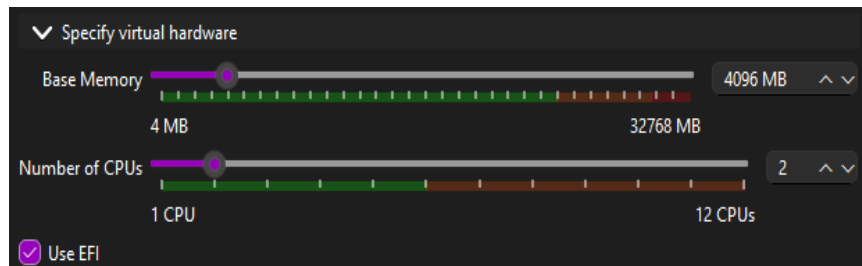
### 6.3 Creación de la máquina virtual

A continuación se detallan los pasos a seguir para la correcta creación de la máquina virtual necesaria para el entorno de desarrollo. Se tiene en cuenta que la imagen del sistema operativo ya se encuentra descargada en el dispositivo.

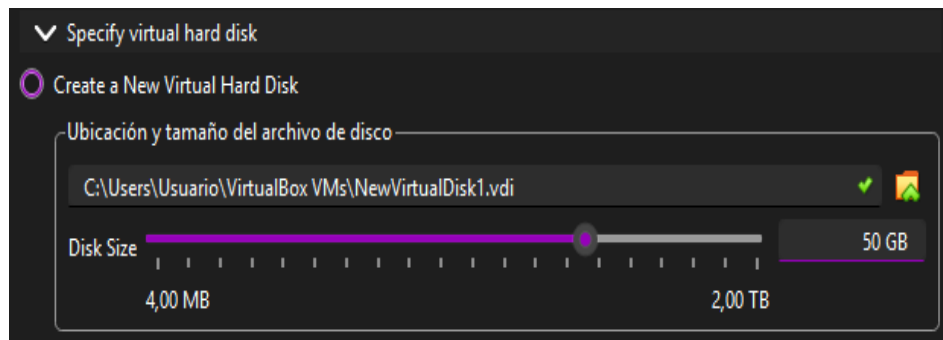
1. Dirigirse al siguiente link (<https://www.virtualbox.org/>) para descargar e instalar el gestor de máquinas virtuales.
2. Una vez instalado, se selecciona la opción **Nueva máquina virtual**.
3. Se abrirá la siguiente ventana en la cual se deben rellenar los campos de nombre e imagen ISO con la imagen del sistema operativo que se va a utilizar.



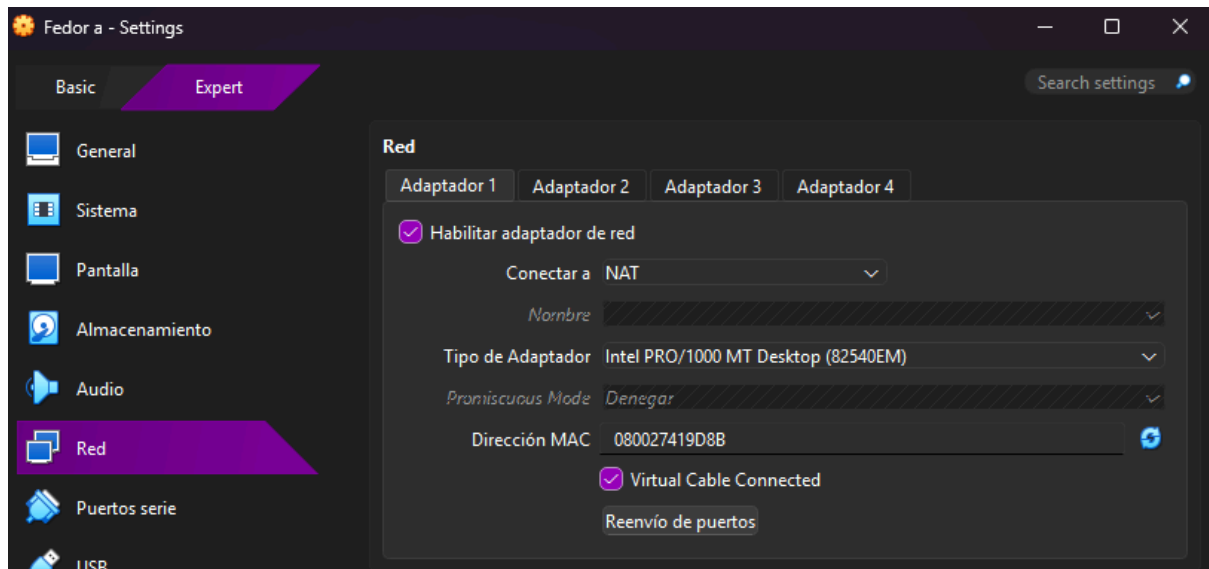
4. Dirigirse a la sección “Specify virtual hardware” donde se seleccionará el número de núcleos y la cantidad de memoria RAM de la máquina, quedando de esta manera:



5. En la sección “specify virtual hard disk” se deberá asignar el espacio que tendrá disponible nuestra máquina virtual.



6. Una vez asignado el disco virtual se debe ingresar a la configuración de la máquina virtual y seleccionar el apartado de Red. En el adaptador 1 se habilita la tarjeta de red y se selecciona la opción NAT.



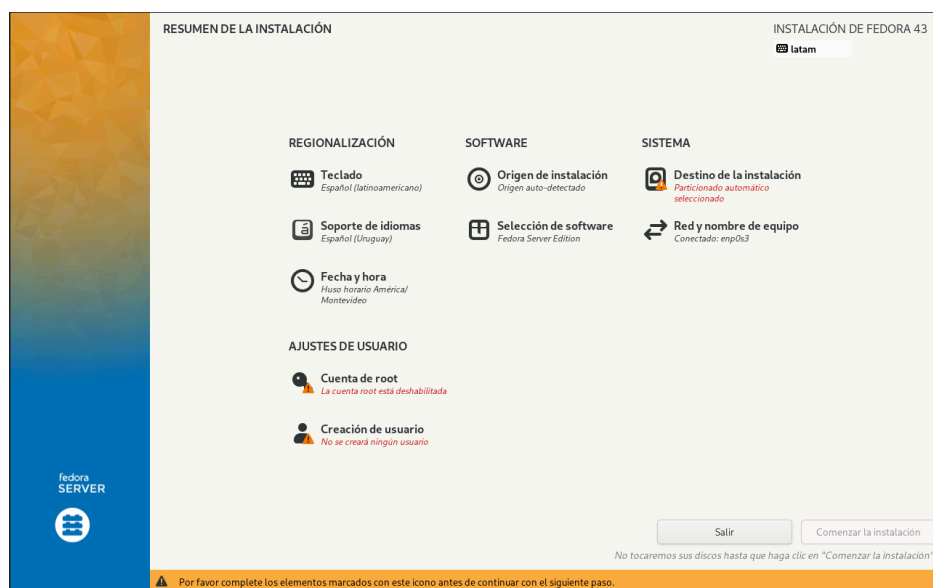
Esta configuración permite que la máquina virtual tenga acceso a internet a través del equipo anfitrión, lo que será necesario para instalar paquetes, descargar herramientas y realizar actualizaciones. Además, al utilizar esta opción la máquina virtual no queda expuesta directamente como equipo dentro de la red local.

Para esta primera etapa se utilizará el adaptador NAT, pero se configurará un adaptador puente para facilitar el acceso mediante SSH en las primeras instancias. Más avanzado el proyecto se configurarán el reenvío de puertos para habilitar el acceso a la máquina virtual a servicios como SSH o HTTP facilitando el acceso pero evitando así la exposición innecesaria de la VM.

## 6.4 Instalación de Fedora Server

Una vez configurada la máquina virtual el siguiente paso es instalar el sistema operativo correspondiente, en este caso Fedora Server 43.

Para iniciar el proceso, se debe ejecutar la máquina virtual y seleccionar la opción **install fedora 43**. Luego de unos segundos se cargará la interfaz gráfica del instalador, desde la cual será necesario seleccionar el idioma del sistema operativo. Esta configuración permitirá continuar con las opciones principales de instalación. (*Fedora Server Interactive Local Installation :: Fedora Docs, 2026*)



A continuación se procederá a configurar las cuentas de usuario del sistema. En primer lugar se creará la cuenta root, que corresponde al usuario con privilegios máximos dentro del sistema operativo. Para ello se debe acceder a la sección **ajustes de usuario > cuenta de root**. En esta pantalla se deberá marcar la opción **activar cuenta de root** e ingresar una contraseña segura en los campos correspondientes. (*Fedora Server Interactive Local Installation :: Fedora Docs, 2026*)

Por motivos de seguridad no se recomienda habilitar el acceso remoto por SSH para el usuario root, ya que esto podría aumentar el riesgo de accesos no autorizados al servidor. La administración remota deberá realizarse preferentemente mediante un



usuario administrador con permisos elevados. (*Fedora Server Interactive Local Installation :: Fedora Docs, 2026*)

CUENTA DE ROOT

Hecho

INSTALACIÓN DE FEDORA 43

latam

La cuenta root se utiliza para administrar el sistema.

El usuario root (también conocido como superusuario) tiene acceso completo a todo el sistema. Por esta razón, es mejor iniciar sesión en este sistema como usuario root solo para realizar el mantenimiento o la administración del sistema.

☐ Desactivar la cuenta de root

Desactivar la cuenta de root bloqueará la cuenta y desactivará el acceso remoto con la cuenta de root. Esto evitará el acceso administrativo involuntario al sistema.

☒ Activar la cuenta de root

Habilitar la cuenta de root le permitirá establecer una contraseña de root y, opcionalmente, habilitar el acceso remoto a la cuenta de root en este sistema.

Contraseña administrativa:

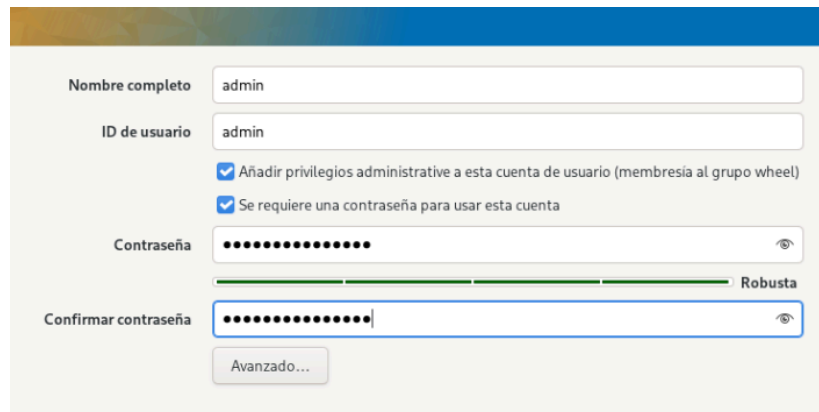
Robusta

Confirmar:

☐ Permitir el acceso SSH de root con contraseña

Una vez configurada la cuenta root se debe regresar al menú principal del instalador para ingresar en la opción **ajustes de usuario > creación de usuario** con el fin de crear a nuestro usuario administrador. (Boy et al., 2026)

En esta sección se deben completar los campos correspondientes al nombre de usuario y contraseña. Además, se debe seleccionar la opción que permite agregar este usuario al grupo de administradores, identificado en fedora como el grupo wheel. De esta forma el usuario podrá ejecutar tareas administrativas mediante el comando **sudo** sin necesidad de utilizar directamente la cuenta root. (*Fedora Server Interactive Local Installation :: Fedora Docs, 2026*)



Nombre completo: admin

ID de usuario: admin

☒ Añadir privilegios administrativos a esta cuenta de usuario (membresía al grupo wheel)

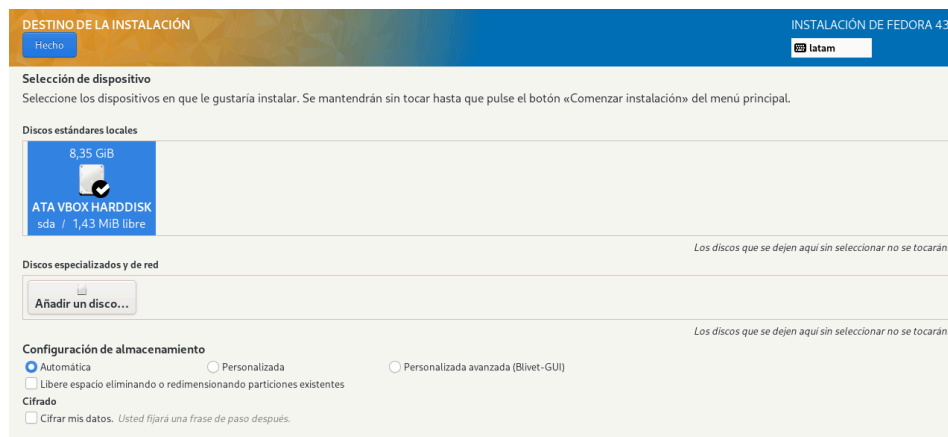
☒ Se requiere una contraseña para usar esta cuenta

Contraseña: [oculto] Robusta

Confirmar contraseña: [oculto]

Avanzado...

Posteriormente, se debe configurar el destino de instalación del sistema operativo. Para ello se debe acceder a la sección **Destino de instalación** y seleccionar el disco virtual asignado previamente a la máquina. Es importante verificar que el disco seleccionado cuente con espacio suficiente para la instalación del sistema y los otros servicios que se utilizarán a lo largo del proyecto. (*Fedora Server Interactive Local Installation :: Fedora Docs, 2026*)



DESTINO DE LA INSTALACIÓN

Hecho

INSTALACIÓN DE FEDORA 43

latam

**Selección de dispositivo**

Seleccione los dispositivos en que le gustaría instalar. Se mantendrán sin tocar hasta que pulse el botón «Comenzar instalación» del menú principal.

**Discos estándares locales**

8,35 GiB

ATA VBOX HARDISK  
sda / 1,43 MiB libre

Los discos que se dejen aquí sin seleccionar no se tocarán.

**Discos especializados y de red**

Añadir un disco...

Los discos que se dejen aquí sin seleccionar no se tocarán.

**Configuración de almacenamiento**

☒ Automática ☐ Personalizada ☐ Personalizada avanzada (Blivet-GUI)

☐ Libere espacio eliminando o redimensionando particiones existentes

**Cifrado**

☐ Cifrar mis datos. Usted fijará una frase de paso después.

Una vez configurada la cuenta root, creada la cuenta de administrador y seleccionado el disco de instalación, el instalador habilitará en el menú principal la opción para comenzar el proceso. Al presionar dicho botón, Fedora Server 43



iniciará la instalación del sistema operativo aplicando las configuraciones definidas anteriormente.

Durante este proceso el instalador copiará los archivos necesarios, configurará el sistema base y creará las cuentas de usuario establecidas. Al finalizar se deberá reiniciar la máquina virtual para iniciar por primera vez Fedora Server desde el disco instalado.

Luego del reinicio el sistema estará listo para iniciar sesión con el usuario administrador creado durante la instalación y continuar con las configuraciones necesarias del servidor.

## 6.5 Configuración de red inicial

Una vez instalado el sistema operativo se debe verificar que el servidor cuente con conexión de red, dado que será necesaria para instalar paquetes, acceder al servidor de forma remota y probar el funcionamiento de la aplicación web desde otros dispositivos.

Para obtener la dirección IP del servidor se utiliza: **ip a**

La dirección IPv4 obtenida será utilizada posteriormente para acceder al servidor desde un navegador web o mediante una conexión SSH.

Es importante comprobar que el servidor tenga salida a internet y que pueda resolver nombres de dominio, para ello se usa el comando **ping** ya sea con una dirección ip como 8.8.8.8 o directamente el nombre de dominio, por ejemplo [google.com](http://google.com).

Para el entorno de pruebas se recomienda utilizar la máquina virtual con un adaptador puente dado que facilita el acceso al servidor desde otros dispositivos conectados a la misma red, siendo extremadamente útil a la hora de probar la aplicación desde distintos dispositivos.

## 6.6 Configuración básica de SSH

Este servicio será utilizado para administrar el servidor de forma remota. Esto permite acceder a la terminal del servidor desde otro dispositivo sin la necesidad de utilizar directamente la consola de la máquina virtual.

En Fedora Server 43 el servicio SSH viene instalado como parte del entorno base del servidor. Por este motivo, en esta etapa no es necesario instalar nuevamente el paquete, sino verificar que el servicio se encuentre disponible y activarlo en caso de que no sea el caso. (Fedora Project, 2026)

Para comprobar el estado del servicio se ejecuta: **systemctl status sshd**

Si el servicio no se encuentra activo se puede iniciar con el siguiente comando: **systemctl enable --now sshd**

Una vez activo el acceso remoto se realiza desde otro equipo mediante el siguiente formato: **ssh usuario@ip\_del\_servidor**, por ejemplo **ssh admin@192.168.1.4**. (Fedora Project, 2026)

Esta configuración permite administrar el servidor de manera cómoda y es una base necesaria para tareas posteriores de mantenimiento, revisión de servicios, actualización del sistema y administración de la aplicación web.

Por seguridad no se recomienda permitir el acceso remoto directo con el usuario root. La administración debe realizarse con un usuario normal con permisos de administrador mediante **sudo**.

## 6.7 Configuración básica del firewall

Fedora Server 43 utiliza **firewalld** como herramienta para administrar el firewall del sistema. La función de este es controlar qué servicios pueden recibir conexiones desde la red, permitiendo así reducir la exposición del servidor y habilitar únicamente los accesos necesarios. (Fedora Project, 2026)

En el contexto del proyecto, se deben permitir los servicios básicos relacionados con la administración remota y el funcionamiento del servidor web mediante XAMPP.

En primera instancia se debe verificar que firewalld esté instalado y activo con el comando **systemctl status firewalld**, si el servicio no se encuentra activo se habilita mediante **systemctl enable --now firewalld**. (Fedora Project, 2026)

Con el firewall ya activado se procede a habilitar el acceso por ssh mediante **firewall-cmd --permanent --add-service=ssh**. Para el resto de servicios como http el comando es idéntico salvo por el valor asignado a la flag **--add-service**. (Fedora Project, 2026)

Una vez agregadas las reglas es necesario recargar la configuración, para ello se usa: **firewall-cmd --reload**.

Finalmente verificamos la configuración aplicada con el comando **firewall-cmd --list-all**. Lo que nos debe devolver los servicios activos del firewall, en este caso SSH, HTTP y Cockpit.

Con esta configuración el servidor permite la administración remota mediante SSH y el acceso web a XAMPP desde otros dispositivos de la red. Al mismo tiempo se mantiene cerrado cualquier otro puerto que no sea necesario para esta etapa del proyecto.

## 6.8 Instalación de XAMPP

Para ejecutar la aplicación web se utilizará XAMPP para linux ya que integra en una sola instalación los servicios necesarios para el entorno inicial: Apache, MariaDB y PHP. Esto permite simplificar la configuración del servidor durante la etapa de pruebas y facilita la verificación del correcto funcionamiento de la aplicación.

El instalador de XAMPP debe descargarse desde el sitio oficial de Apache Friends. El archivo correspondiente para Linux suele tener extensión .run

Por ejemplo, en este momento el comando para descargar la versión 8.2.12 es el siguiente:

**wget**

<https://sourceforge.net/projects/xampp/files/XAMPP%20Linux/8.2.12/xampp-linux-x64-8.2.12-0-installer.run> (Mutai, 2026)

Se recomienda crear una carpeta de descargas en la cual posicionarse antes de ejecutar el comando anterior.

Una vez completada la descarga se creará en nuestra carpeta un instalador con un nombre similar a este: **xampp-linux-x64-8.2.12-0-installer.run**

Antes de ejecutarlo se deben asignar permisos con el comando **chmod +x** y el nombre del archivo. Una vez asignados se procede a iniciar el instalador con

permisos de administrador: **sudo ./xampp-linux-x64-8.2.12-0-installer.run** (Mutai, 2026)

Durante el proceso de instalación se deben aceptar las opciones indicadas por el asistente y al finalizar XAMPP quedará instalado por defecto en **/opt/lampp**

Esta ubicación será utilizada posteriormente para iniciar los servicios pertinentes, ubicar archivos del proyecto y verificar el funcionamiento de Apache, PHP y MariaDB. (Mutai, 2026)

Es importante aclarar que XAMPP se utilizará únicamente en el entorno de desarrollo y pruebas del proyecto por su facilidad para integrar los paquetes necesarios en una sola descarga. Sin embargo, no se plantea su uso en un entorno de producción final. En tal caso, y si se llega a concretar un uso formal de los servidores del departamento técnico de informática del hospital, se recomienda instalar los componentes por separado (Apache HTTP server, PHP y MariaDB). De esta manera, cada servicio se podrá configurar, actualizar y administrar de forma independiente, aplicando mejores prácticas de seguridad, mantenimiento y control del servidor.

## 6.9 Instalación y configuración de Node.js

Para trabajar con React, Vite y Material UI, será necesario instalar Node.js. Se recomienda utilizar una versión LTS, ya que estas versiones están pensadas para mayor estabilidad.

Pasos generales de instalación:

1. Descargar Node.js desde el sitio oficial.



2. Instalar la versión LTS.
3. Abrir una terminal.
4. Verificar la instalación con: `node -v`
5. Verificar npm con: `npm -v`

Si ambos comandos muestran una versión instalada, la configuración inicial de Node.js fue realizada correctamente.

Node.js será utilizado únicamente para el entorno de desarrollo del frontend. El backend principal del sistema será desarrollado en PHP.

## **6.10 Configuración de Visual Studio Code**

Visual Studio Code será configurado con extensiones que faciliten el desarrollo del proyecto, tanto en el backend como en el frontend.

El objetivo es que todos los integrantes del equipo trabajen con un entorno similar, reduciendo diferencias de formato, errores de configuración y problemas al ejecutar el proyecto.

### **6.10.1 PHP Intelephense**

PHP Intelephense será utilizado para mejorar el desarrollo en PHP. Esta extensión proporciona autocompletado, detección de errores, navegación entre archivos y análisis básico del código.

Su uso facilita la escritura del backend, especialmente en archivos relacionados con conexión a base de datos, validaciones y procesamiento de formularios.





### **6.10.2 GitLens**

GitLens será utilizado para mejorar la integración de Git dentro de Visual Studio Code. Permite visualizar el historial de cambios, autores de cada modificación, ramas y commits directamente desde el editor.

Esta extensión resulta útil para el trabajo colaborativo, ya que facilita identificar qué integrante realizó cada cambio y en qué momento.

### **6.10.3 Prettier**

Prettier será utilizado para mantener un formato de código uniforme en el frontend y, cuando corresponda, en otros archivos del proyecto.

Su uso permite aplicar automáticamente reglas de formato, evitando diferencias visuales entre archivos editados por distintos integrantes del equipo. Esto mejora la legibilidad del código y facilita su mantenimiento.

### **6.10.4 Extensión MySQL o Database Client**

Se recomienda instalar una extensión de MySQL o Database Client para poder conectarse a la base de datos directamente desde Visual Studio Code.

Esto permitirá consultar tablas, visualizar registros y probar consultas SQL sin necesidad de salir del editor. Aunque la consola de MariaDB seguirá siendo una herramienta válida, contar con acceso a la base de datos desde VS Code puede agilizar el desarrollo.

### 6.10.5 Extensiones recomendadas para React

Para trabajar con React se recomienda utilizar extensiones relacionadas con JavaScript, JSX y componentes. Estas extensiones facilitan el autocompletado, la detección de errores y la generación rápida de estructuras de código.

Extensiones recomendadas:

- React Developer Tools en el navegador.
- ESLint en Visual Studio Code.

## 6.11 Verificación del entorno de desarrollo

### 6.11.1 Verificación de Apache

Apache se puede verificar accediendo desde un navegador en el anfitrión a la dirección IP del servidor. Un ejemplo de esto sería `http://192.168.1.4`, si al ingresar a esta dirección se muestra la página inicial de XAMPP se confirma que el servidor web Apache está activo y funciona correctamente.

También se puede realizar una prueba desde la propia terminal a través del comando `curl -i http://localhost`, este comando devuelve únicamente las cabeceras de la respuesta HTTP. Si Apache funciona correctamente se debería ver algo parecido a la siguiente imagen.

```
admin@localhost:~$ curl -i http://192.168.1.4
HTTP/1.1 302 Found
Date: Wed, 27 May 2026 20:28:20 GMT
Server: Apache/2.4.58 (Unix) OpenSSL/1.1.1w PHP/8.2.12 mod_perl/2.0.12 Perl/v5.34.1
X-Powered-By: PHP/8.2.12
Location: http://192.168.1.4/dashboard/
Content-Length: 0
Content-Type: text/html; charset=UTF-8
```

### 6.11.2 Verificación de PHP

Para comprobar que PHP se ejecuta correctamente se creará un archivo de prueba dentro del directorio público de XAMPP, siendo la dirección de este **/opt/lampp/htdocs**. Para ello se crea un archivo **info.php** utilizando vim mediante el siguiente comando: **vim /opt/lampp/htdocs/info.php**, puede ser necesario utilizar privilegios de superusuario para poder crear el documento.

Una vez en el archivo ingresamos el siguiente código:

```
<?php
phpinfo()
?>
```

Después se guarda el archivo y se procede a acceder desde el navegador de nuestro equipo anfitrión mediante el siguiente formato de URL: **http://IP\_DEL\_SERVIDOR/info.php**, por ejemplo <http://192.168.1.4/info.php>.

Si PHP funciona correctamente se mostrará una página con información técnica sobre la versión instalada y sus módulos activos.

Una vez finalizada la comprobación es importante eliminar este archivo dado que expone información interna del servidor.

### 6.11.3 Verificación de MariaDB

Para la verificación de MariaDB es necesario acceder a la consola del motor de base de datos, en esta prueba se accederá con el usuario root (el cual no es el

mismo usuario root que gestiona el sistema operativo). Para ello se ejecuta **sudo /opt/lampp/bin/mysql -u root**.

Si el acceso es correcto se abrirá la siguiente consola:

```
admin@localhost:~$ sudo /opt/lampp/bin/mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 10.4.32-MariaDB Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Desde la cual se podrán ejecutar consultas SQL como **SHOW DATABASES**; la cual mostrará las bases de datos disponibles dentro del sistema demostrando así el correcto funcionamiento de MariaDB.

Con estas pruebas queda demostrado que XAMPP y sus servicios principales se encuentran operativos y disponibles.

#### 6.11.4 Verificación de Node.js

Para verificar Node.js y npm, se deben ejecutar los siguientes comandos:

node -v

npm -v

Si ambos comandos muestran una versión instalada, significa que Node.js y npm están correctamente configurados.

### 6.11.5 Verificación de Git

Para verificar Git, se ejecuta:

```
git --version
```

Luego se comprueba la configuración del usuario:

```
git config --global --list
```

### 6.12 Clonado del repositorio del proyecto

Una vez instalado Git y verificada su configuración, el siguiente paso consiste en clonar el repositorio del proyecto desde GitHub. Esto permitirá obtener una copia local del código fuente, la documentación técnica, los scripts de base de datos y los recursos necesarios para trabajar sobre el sistema S.I.G.S.M.

Para clonar el repositorio, primero se debe abrir una terminal y ubicarse en la carpeta donde se desea guardar el proyecto. Luego se ejecuta el siguiente comando:

```
git clone [URL_DEL_REPOSITORIO]
```

Por ejemplo:

```
git clone https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU
```

Una vez finalizado el proceso, se ingresará a la carpeta del proyecto mediante:

```
cd sigsm
```

Es importante destacar que en el directorio base del proyecto se encuentran las carpetas de primera, segunda y tercera entrega, así como una carpeta llamada

prototipos a la cual deberemos acceder si se quieren visualizar. Podemos hacerlo mediante el siguiente comando: `cd prototipos/frontend`.

Luego de clonar el repositorio, se recomienda verificar el estado del proyecto con:  
`git status`

Si el repositorio fue clonado correctamente, Git mostrará el estado actual de la rama y confirmará que el directorio pertenece a un repositorio válido.

También se puede verificar la conexión con el repositorio remoto mediante: `git remote -v`

Este comando debe mostrar la URL del repositorio de GitHub asociada al proyecto. De esta forma se confirma que la copia local está vinculada correctamente con el repositorio remoto.

A partir de este punto, cada integrante del equipo podrá trabajar sobre el proyecto, realizar modificaciones, crear commits y subir los cambios al repositorio de GitHub siguiendo las convenciones definidas por el equipo.

### **6.13 Pruebas y evidencias de la instalación**

Si bien se ha ido dejando evidencia de la correcta instalación de los paquetes y herramientas necesarios para el funcionamiento del servidor durante el presente manual, esta sección tiene como objetivo unificar en un solo lugar dichas evidencias para facilitar la revisión final del entorno.

La idea de esta sección es comprobar de manera ordenada, que todo lo instalado hasta este punto funciona correctamente. Para ello se realizarán pruebas básicas

sobre el sistema operativo, la conexión de red, SSH, firewall, XAMPP, Apache; PHP, MariaDB y Git.

Cada prueba deberá acompañarse de su respectiva captura de pantalla del resultado obtenido. Esto permite demostrar no solo que el servidor fue instalado sino que también se encuentra funcional y disponible para continuar con el despliegue del sistema.

- **Versión del sistema operativo:** se verifica con el comando `cat /etc/fedora-release`. El resultado esperado es que muestre Fedora 43

```
admin@localhost:~$ cat /etc/fedora-release
Fedora release 43 (Forty Three)
```

- **Conectividad de red:** se comprueba con el comando `ping 8.8.8.8`. El resultado esperado es recibir respuestas correctamente, lo que confirma que el servidor tiene conexión a internet.

```
admin@localhost:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes de datos.
64 bytes desde 8.8.8.8: icmp_seq=1 ttl=116 tiempo=13.3 ms
64 bytes desde 8.8.8.8: icmp_seq=2 ttl=116 tiempo=13.1 ms
```

- **Dirección IP del servidor:** se obtiene mediante el comando `ip a`. El resultado esperado es identificar una dirección IPv4 válida asignada al servidor.

```
admin@localhost:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:41:9d:8b brd ff:ff:ff:ff:ff:ff
    altname enx080027419d8b
    inet 192.168.1.4/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
        valid_lft 71837sec preferred_lft 71837sec
    inet6 2000:a4:1fee:6d00:a00:27ff:fe41:9d8b/64 scope global dynamic noprefixroute
        valid_lft 26914sec preferred_lft 26914sec
    inet6 fe80::a00:27ff:fe41:9d8b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

- **Estado de SSH:** se verifica con **systemctl status sshd**. El resultado esperado es que el servicio figure como activo.

```
admin@localhost:~$ systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf
   Active: active (running) since Wed 2026-05-27 15:35:39 -03; 4h 12min ago
   Invocation: 14192bd2029743c8909c7bcbcd925472
   Docs: man:sshd(8)
         man:sshd_config(5)
   Main PID: 842 (sshd)
   Tasks: 1 (limit: 10669)
   Memory: 1.6M (peak: 2.2M)
   CPU: 21ms
   CGroup: /system.slice/sshd.service
           └─842 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

may 27 15:35:39 localhost.localdomain systemd[1]: Starting sshd.service - OpenSSH server daemon...
may 27 15:35:39 localhost.localdomain sshd[842]: Server listening on 0.0.0.0 port 22.
may 27 15:35:39 localhost.localdomain sshd[842]: Server listening on :: port 22.
may 27 15:35:39 localhost.localdomain systemd[1]: Started sshd.service - OpenSSH server daemon.
```

- **Estado del firewall:** se comprueba con **systemctl status firewalld**. El resultado esperado es que el servicio se encuentre activo.

```
admin@localhost:~$ systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; enabled; preset: enabled)
   Drop-In: /usr/lib/systemd/system/service.d
            └─10-timeout-abort.conf
   Active: active (running) since Wed 2026-05-27 15:35:39 -03; 4h 14min ago
   Invocation: 276e46863364467b8e55d54fbb5422c6
   Docs: man:firewalld(1)
   Main PID: 773 (firewalld)
   Tasks: 4 (limit: 10669)
   Memory: 48.9M (peak: 49.4M)
   CPU: 81ms
   CGroup: /system.slice/firewalld.service
           └─773 /usr/bin/python3 -sP /usr/bin/firewalld --nofork --nopic

may 27 15:35:37 localhost systemd[1]: Starting firewalld.service - firewalld - dynamic firewall daemon...
may 27 15:35:39 localhost systemd[1]: Started firewalld.service - firewalld - dynamic firewall daemon.
may 27 16:23:10 192.168.1.4 firewalld[773]: WARNING: ALREADY_ENABLED: ssh
```

- **Reglas del firewall:** se revisan con **sudo firewall-cmd --list-all**. El resultado esperado es que se encuentren habilitados como mínimo los servicios HTTP y SSH.



```
admin@localhost:~$ sudo firewall-cmd --list-all
[sudo] contraseña para admin:
FedoraServer (default, active)
  target: default
  ingress-priority: 0
  egress-priority: 0
  icmp-block-inversion: no
  interfaces: enp0s3
  sources:
  services: cockpit dhcpv6-client http https ssh
  ports:
  protocols:
  forward: yes
  masquerade: no
  forward-ports:
  source-ports:
  icmp-blocks:
  rich rules:
```

- **Estado de XAMPP:** se verifica con `sudo /opt/lampp/lampp status`. El resultado esperado es que Apache y MariaDB se encuentren activos.

```
admin@localhost:~$ sudo /opt/lampp/lampp status
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
Version: XAMPP for Linux 8.2.12-0
egrep: warning: egrep is obsolescent; using grep -E
Apache is running.
egrep: warning: egrep is obsolescent; using grep -E
MySQL is running.
egrep: warning: egrep is obsolescent; using grep -E
ProFTPD is running.
```

- **Respuesta de Apache:** se comprueba con `curl -I http://localhost`. El resultado esperado es obtener una respuesta HTTP válida.



- **Verificación de MariaDB:** se comprueba con **sudo /opt/lampp/bin/mysql -u root**. El resultado esperado es que se abra la consola de MariaDB.

```
admin@localhost:~$ sudo /opt/lampp/bin/mysql -u root
[sudo] contraseña para admin:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 10.4.32-MariaDB Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

- **Verificación de Git:** se verifica mediante el comando **git --version**. El resultado esperado es que se muestre la versión instalada de Git.

```
admin@localhost:~$ git --version
git version 2.54.0
```

- **Verificación de node, npm, git y Visual Studio Code:** Se verifica mediante los comandos: **node -v**, **npm -v**, **git --version** y **code --version**. El resultado esperado es que muestre las versiones instaladas de cada elemento.

```
PS C:\Users\Usuario\Desktop\stuff\utu\tercero\Proyecto\prototipos> node -v
v22.18.0
PS C:\Users\Usuario\Desktop\stuff\utu\tercero\Proyecto\prototipos> npm -v
11.5.2
PS C:\Users\Usuario\Desktop\stuff\utu\tercero\Proyecto\prototipos> git --version
git version 2.43.0.windows.1
PS C:\Users\Usuario\Desktop\stuff\utu\tercero\Proyecto\prototipos> code --version
1.125.1
```

## 7. Conclusión

A partir del desarrollo de esta primera entrega se establecieron las bases técnicas iniciales para la construcción del sistema S.I.G.S.M., orientado a dar respuesta a necesidades concretas del Hospital de Clínicas mediante dos módulos principales:

la gestión y acceso digital a documentación para pacientes, y la gestión y seguimiento de traslados mediante ambulancias u otros vehículos.

En primer lugar, se realizó la justificación tecnológica del stack seleccionado, considerando las características del sistema, el contexto institucional y la necesidad de contar con una solución web organizada, mantenible y compatible con un entorno de servidor GNU/Linux. Se definió el uso de PHP para el backend, MySQL/MariaDB para la persistencia de datos, React y Material UI para el desarrollo de la interfaz, Vite y Node.js para el entorno frontend, Git/GitHub para el control de versiones y Visual Studio Code como entorno principal de desarrollo.

También se avanzó en la definición visual del sistema mediante prototipos de interfaz de usuario. Estos prototipos permitieron representar las pantallas principales de ambos módulos, diferenciando las vistas administrativas utilizadas por funcionarios del hospital y las vistas destinadas a pacientes mediante acceso móvil por código QR. Esta etapa permitió anticipar la organización de la información, la estructura de navegación y los criterios generales de experiencia de usuario.

En relación con el diseño responsivo, se planteó una estrategia orientada a adaptar las interfaces a distintos dispositivos. Esto resulta especialmente importante debido a que el sistema será utilizado tanto desde equipos de escritorio, para tareas administrativas, como desde celulares, en el caso de pacientes que accedan a documentación o encuestas mediante QR. Para ello, se propuso el uso de React, Material UI, Flexbox, Grid y componentes adaptables que permitan mantener una experiencia clara y coherente.

Asimismo, se desarrolló el modelado inicial de datos del sistema, identificando las entidades principales, relaciones, cardinalidades, restricciones no estructurales y el esquema relacional normalizado. Este trabajo permitió organizar la información

correspondiente a documentos, categorías, encuestas, respuestas, usuarios, traslados, vehículos, funcionarios, ubicaciones y estados. El modelo planteado servirá como base para la futura implementación de la base de datos y podrá ajustarse en próximas etapas según los avances del proyecto.

Por otra parte, se documentó la configuración del entorno de desarrollo, integrando las herramientas necesarias para ejecutar y probar el sistema. Se contempló el uso de una máquina virtual con Fedora Server, la instalación de XAMPP, Apache, PHP, MariaDB, Git, Node.js, Vite y Visual Studio Code, además de pruebas verificables para comprobar el funcionamiento del entorno. Esta documentación resulta importante porque permite replicar el ambiente de trabajo y mantener condiciones similares entre desarrollo, pruebas y futura instalación.

En conclusión, esta primera entrega permitió consolidar una base técnica y organizativa para el desarrollo del sistema S.I.G.S.M. Si bien el sistema aún se encuentra en una etapa inicial, ya se definieron las tecnologías, el enfoque visual, la estructura responsiva, el modelo de datos y el entorno de desarrollo. Estos elementos serán fundamentales para continuar con la implementación funcional de los módulos en las próximas entregas del proyecto.

## 8. Bibliografía

Fedora Project. (2026, april 28). *Fedora Server Installation Guide*. Fedora Server Installation Guide.

<https://docs.fedoraproject.org/en-US/fedora-server/installation/>

Fedora Project. (2026, april 28). *Post Installation Tasks*. Post Installation Tasks.

<https://docs.fedoraproject.org/en-US/fedora-server/installation/postinstallation-tasks/>

Git. (n.d.). *Learn*. Git. Retrieved June 23, 2026, from <https://git-scm.com/learn>

Git. (n.d.). *Reference*. Git. Retrieved June 22, 2026, from <https://git-scm.com/docs>

MariaDB. (n.d.). *ACID: Concurrency Control with Transactions*. ACID: Concurrency Control with Transactions.

<https://mariadb.com/docs/general-resources/database-theory/acid-concurrency-control-with-transactions>

MariaDB. (2026, June 4). *Foreign Keys | Server*. MariaDB. Retrieved June 22, 2026, from

<https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/optimization-and-indexes/foreign-keys>

Material UI. (n.d.). MUI: The React component library you always wanted. Retrieved June 22, 2026, from <https://mui.com/>

Mutai, J. (2026, april 10). *Install XAMPP on Fedora 42 / 41 (All PHP Editions)*. Install XAMPP on Fedora 42 / 41 (All PHP Editions).

<https://computingforgeeks.com/how-to-install-xampp-on-fedora/>

Node.js. (n.d.). *Learn Node.js*. Node.js. Retrieved June 22, 2026, from <https://nodejs.org/learn>

PHP. (n.d.). PHP.net. Retrieved June 22, 2026, from <https://www.php.net/>



Pokojny, P. (n.d.). *PDO - Manual*. PHP. Retrieved June 22, 2026, from <https://www.php.net/manual/en/book.pdo.php>

*Preface - Manual*. (2026, June 23). PHP. Retrieved June 23, 2026, from <https://www.php.net/manual/en/>

React. (n.d.). React. Retrieved June 22, 2026, from <https://react.dev/>

Visual Studio Code. (n.d.). *Visual Studio Code - The open source AI code editor | Your home for multi-agent development*. Visual Studio Code - The open source AI code editor | Your home for multi-agent development. Retrieved June 22, 2026, from <https://code.visualstudio.com/>

Vite.dev. (n.d.). *Getting Started*. Vite. Retrieved June 22, 2026, from <https://vite.dev/guide/>

XAMPP. (n.d.). XAMPP Installers and Downloads for Apache Friends. Retrieved June 22, 2026, from <https://www.apachefriends.org/es/index.html>